

归 档 号：H232-2025-JX17-103

武汉工商学院毕业论文(设计)

学院：计算机与自动化学院

专业：计算机科学与技术 年级：2021 级

学生：吴丰宝 学号：1952201125

指导教师：朱安/李娟 职称：工程师/无

题目：基于 SpringBoot 的二次元社交平台的设计与实现

2025 年 5 月 12 日

武汉工商学院

本科毕业论文（设计）原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名：吴丰宝

2025 年 5 月 12 日

目 录

摘 要	1
关键词	1
Abstract	2
Key words	2
1 绪论	3
1.1 前言	3
1.2 选题背景	3
1.3 选题目的	3
1.4 选题意义	4
2 相关技术和理论概述	5
3 系统功能分析	7
3.1 系统功能介绍	7
3.2 系统功能模块分析	7
4 系统分析与设计	10
4.1 总体架构设计	10
4.2 数据库设计	11
4.2.1 数据库概念模型的设计	11
4.2.2 概念模型转为关系模型	16
4.2.3 数据库表的设计	17
5 系统的开发设计与实现	20
5.1 登录模块	20
5.1.1 登录模块界面设计	20
5.1.2 登录模块功能设计	20
5.1.3 登录模块核心代码	22
5.2 用户管理模块	23
5.2.1 用户管理模块界面设计	23
5.2.2 用户管理模块功能设计	23
5.2.3 用户管理模块核心代码	24

5.3	作品管理模块	25
5.3.1	作品管理模块界面设计	25
5.3.2	作品管理模块功能设计	25
5.3.3	作品管理模块核心代码	26
5.4	评论管理模块	27
5.4.1	评论管理模块界面设计	27
5.4.2	评论管理模块功能设计	28
5.4.3	评论管理模块核心代码	28
5.5	公告管理模块	29
5.5.1	公告管理模块界面设计	29
5.5.2	公告管理模块功能设计	30
5.5.3	公告管理模块核心代码	30
5.6	首页模块	32
5.6.1	首页模块界面设计	32
5.6.2	首页模块功能设计	32
5.6.3	首页模块核心代码	33
5.7	消息模块	34
5.7.1	消息模块界面设计	34
5.7.2	消息模块功能设计	34
5.7.3	消息模块核心代码	35
5.8	收藏模块	35
5.8.1	收藏模块界面设计	35
5.8.2	收藏模块功能设计	36
5.8.3	收藏模块核心代码	36
5.9	历史记录模块	37
5.9.1	历史记录模块界面设计	37
5.9.2	历史记录模块功能设计	38
5.9.3	历史记录模块核心代码	38
5.10	投稿管理模块	39
5.10.1	投稿管理模块界面设计	39

5.10.2 投稿管理模块功能设计	40
5.10.3 投稿管理模块核心代码	40
5.11 个人主页模块	41
5.11.1 个人主页模块界面设计	41
5.11.2 个人主页管理模块功能设计	42
5.11.3 个人主页模块核心代码	42
6 系统功能测试	44
6.1 测试方法	44
6.2 测试环境	44
6.3 功能测试	44
6.3.1 登录模块功能测试	44
6.3.2 用户管理模块功能测试	45
6.3.3 作品管理模块功能测试	45
6.3.4 评论管理模块功能测试	45
6.3.5 公告管理模块功能测试	46
6.3.6 首页模块功能测试	46
6.3.7 消息模块功能测试	46
6.3.8 收藏模块功能测试	47
6.3.9 历史记录模块功能测试	47
6.3.10 投稿模块功能测试	47
6.3.11 个人主页模块功能测试	48
7 总结	49
参考文献	50
致谢	52

基于 SpringBoot 的二次元社交平台的设计与实现

摘 要

当今互联网技术发展迅速，普及广泛，使用网络社交平台已经成为人们日常生活中最稀疏平常的一部分。同时，近些年二次元文化作为一种独特的亚文化，在全球范围内迅速崛起，尤其在年轻一代中广受欢迎。

本文系统地介绍了二次元社交平台的设计与开发的全过程，首先在前期的系统需求分析中明确功能定位、开发平台的选择以及开发环境的搭建，随后中期进行数据库数据表的设计、系统架构的搭建、代码的编写，最后到了多轮的系统功能的测试与修改。本系统中包括用户界面和管理员界面，管理员界面主要有登录、二次元作品管理、用户信息管理、公告管理、评论管理等功能，用户界面主要有登录和注册、个人信息管理、二次元作品投稿、评论、收藏、历史记录管理等功能。

本项目中使用 Mysql 作为后台数据库，把 Apache Tomcat 作为应用服务器，前端结合 Vue 实现网页的搭建以及美化处理，后端采取了 SpringBoot 框架进行系统设计，代码开发工具是 IntelliJ IDEA。

关键词：二次元社交平台；SpringBoot；Vue

Design and implementation of two-dimensional social platform based on SpringBoot

Abstract

Nowadays, Internet technology is developing rapidly and widely. Using online social platforms has become the most common part of people's daily lives. At the same time, in recent years, as a unique subculture, the secondary culture has risen rapidly in the world, especially among the young generation.

This article systematically introduces the entire process of the design and development of a two-dimensional social platform. Firstly, in the early stage of system requirement analysis, the functional positioning, the selection of the development platform, and the setup of the development environment are clarified. Then, in the middle stage, the design of database data tables, the construction of the system architecture, and the writing of code are carried out. Finally, multiple rounds of system function testing and modification are conducted. This system includes a user interface and an administrator interface. The administrator interface mainly has functions such as login, management of two-dimensional works, user information management, announcement management, and comment management. The user interface mainly has functions such as login and registration, personal information management, submission of two-dimensional works, comments, collections, and management of historical records.

In this project, MySQL is used as the backend database, Apache Tomcat is adopted as the application server, and Vue is combined with the front end to build and beautify the web pages. The SpringBoot framework is used for system design in the back end, and IntelliJ IDEA is the code development tool.

Key words: the two-dimensional social platform; SpringBoot; Vue

1 绪论

1.1 前言

现代社会互联网技术发展迅速，普及广泛，无论是国内还是国外都有各种各样的社交平台供用户们的选择也越来越丰富，但是这类平台往往呈现的内容丰富多彩，没有完全针对某一类用户群体去进行内容投放，从而容易让用户时常看不到自己想看的内容。其中国外二次元用户群体数量近些年越来越庞大，年轻人更是占比近 70%，他们对于内容的需求也是丰富多样，而一些传统社交平台还是无法满足。针对二次元用户，开发一个能够有助于那些拥有共同兴趣爱好年轻人之间相互交流，寻找一个共鸣的社交平台意义非凡^[1]。

在开发二次元社交平台的系统时，如果使用传统的开发方式会出现代码耦合度高、代码之间关联性弱、代码冗余等一系列加重开发效率的问题，因此本文采用的是结合 SpringBoot 框架开发后端^[2]。同时前端结合 Vue 去进行开发，这使得开发效率成倍提高，也确保了系统的稳定、高效与可扩展性^[3]。

1.2 选题背景

本系统的开发主要是考虑现如今越来越多二次元用户群体的增加，传统社交平台无法满足这类用户的特定需求的背景下构思出来的。二次元用户期望一个高度沉浸式的社交环境，这在传统社交平台上很难实现。该系统的开发设计完全可以适用于部分想看和二次元相关的作品的用户^[4]。而且也考虑到传统社交平台作品种类丰富，很难一直看到二次元相关的作品，如基于动漫 IP 的相关作品，所以需要一点特点明显，针对性强的社交平台。

二次元社交平台上可以看到很多创作者发布的二次元作品，看到这些往往能让他们身心得到放松，同时，参与同人创作，如绘画、写作、视频等活动也有利于培养个人才能、提高自信心，共同搭建一个二次元爱好者聚集在一起的社交平台。

1.3 选题目的

不同于传统的社交平台，二次元社交平台往往围绕共同的兴趣爱好形成紧密联系的小圈子。成员之间相互支持、分享心得，并积极参与各种线上的社区活动。这种基于兴趣而形成的社群关系，不仅增强了个体归属感，还促进了积极健康的社交互动模式。在全球化的背景下，二次元文化打破了国界限制，成为连接世界各地爱好者之间的纽带。例如日本动漫在中国拥有庞大的受众群体；同样地，中

国原创 IP 也在海外市场崭露头角。

综上所述，通过个性化推荐和游戏化机制，给用户提供专属二次元的乐趣，增强用户粘性。同时鼓励用户创造和分享原创内容，从自制视频到图文创作，共同搭建一个专属于二次元用户的社交平台。

1.4 选题意义

随着二次元文化在全球范围内的快速传播，以动画、漫画、游戏（ACG）为核心的亚文化社群规模不断扩大，年轻用户对专属社交空间的需求日益增长。然而，传统的社交平台（如小红书、微博）缺乏针对二次元用户的深度定制功能^[5]。

二次元用户群体具有鲜明的文化特征和行为习惯，如同好互动、同人创作等。通过分析这一特定用户群体的需求，可以深化对兴趣社交、沉浸式体验等领域的理论研究。其次，本研究能够探索亚文化社群的线上组织模式。拥有共同兴趣爱好年轻人之间会聚集在一起相互交流，共同创造和分享原创二次元内容，从自制视频到图文创作，形成了一个活跃的内容创作者生态。这不仅促进了 UGC（用户生成内容）的发展，也为许多有才华的年轻人提供了展示自我的舞台。最后，该研究可为其他垂直社区（如传统文化、电竞等）的构建提供参考^[6]。

2 相关技术和理论概述

随着越来越多代码工具的出现，选择适合自己开发项目的一套软件和相关技术对于开发者来说至关重要。本系统使用 IntelliJ IDEA 开发软件开发前后端代码。前端采用 Vue 框架进行页面的快速搭建^[7]。后端 Java 采用 Spring Boot 框架，利用其自动配置特性和起步依赖机制能够显著提升开发效率，特别适合快速迭代的社交类应用开发^[8]。数据库采用的是 MySQL 数据库，凭借着开源免费、支持高并发、优化查询等特性，在 web 应用程序上使用效果显著。服务器选择了同样是免费好用的 Tomcat 服务器，稳定的性能深受广大 Java 开发者喜爱^[9]。

(1) IntelliJ IDEA 介绍

本系统是在 IntelliJ IDEA 开发软件中设计出来的^[10]。IntelliJ IDEA 是由 JetBrains 公司开发的一款强大的 Java 集成开发环境，它拥有专业 Java 开发所需要的一切功能，包括智能代码编辑器、内置开发着工具，以及对框架、Web 开发等的支持^[11]。尤其在在智能代码助手、代码自动提示、重构、JavaEE 支持、各类版本工具(git、svn 等)、JUnit、代码分析、创新的 GUI 设计等方面的功能可以说是超常的^[12]。它能够利用所有必要工具消除常规障碍，更快地将想法变为现实。在编写代码上，具有上下文敏感的代码补全，提供仅对当前文本光标位置有效的建议。开启高效编码，能更快地看到结果，并专注于构建软件^[13]。

(2) MySQL 介绍

MySQL 是一个对外开源且免费的关系型数据库管理系统，由国外的一家出名公司 MySQL AB 开发，后被大家现在所熟知的 Oracle 收购^[14]。它采用的是不同于一般程序语言的 SQL（结构化查询语言）进行数据管理，秉持着开源免费、支持高并发、优化查询、跨平台、支持事务、可拓展性强等特点广泛应用于 Web 应用、企业级系统等领域^[15]。同时 MySQL 还在持续更新，新增了如原生 JSON 支持、多源复制等高级功能，进一步提升了其性能和灵活性。

(3) Tomcat 介绍

Tomcat 是国外的 Apache 软件基金会开发的一个核心项目，它是由包括 Sun 公司和其他一些公司及个人共同开发而成^[16]。本系统的后端使用的是 Tomcat 8.5 版本，性能稳定，深受 Java 爱好者们的喜爱^[17]。同时它是一个免费的开放源代码的 Web 应用服务器，属于轻量级应用服务器，在中小型系统和并发访问用户不是很多的场合下被普遍使用，是开发和调试 JSP 程序的首选^[18]。

(4) Vue 介绍

Vue.js 相对于其他框架，它是一款用于构建用户界面的渐进式 JavaScript 框架^[19]。它的本质还是基于 JavaScript 和 html 构建而成，最重要的意义是为开发着提供了一套声明式的、代码组件化的编程模型，帮助开发者高效地进行界面设计。Vue 的底层设计为可以自下向上逐层应用，分为视图层，模型层，连接视图和数据的中间件。Vue 的核心在于专注视图层，方便观察界面设计，同时也便于与其他项目整合。当与现代化的插件工具结合使用时，Vue 也完全能够流畅使用。

(5) SpringBoot 介绍

本系统的后端是基于 SpringBoot 框架来开发应用程序，它旨在帮助初学的开发者简化 Java 应用程序的搭建和配置。其中 SpringBoot 采用“约定优于配置”（Convention Over Configuration）的理念，通过自动配置和起步依赖（Starters）的方式来简化 Spring 应用的开发^[20]。SpringBoot 会根据项目依赖自动配置 Bean，会提供一系列预定义的依赖包，通过它们的帮忙，避免手动管理版本冲突，可以通过 application.properties 或 application.yml 文件灵活配置连接数据库的一些必要参数信息^[21]。

3 系统功能分析

3.1 系统功能介绍

本系统分为管理员和用户两种用户类型，二次元社区系统也因此分为二次元社区和二次元社区后台管理两个模块，用户只能在二次元社区模块，管理员只能在二次元社区后台管理模块。

用户功能：用户登录二次元社区系统后，可以进入主页浏览作品，查看消息，管理个人主页，投稿作品，管理收藏，管理历史记录。

管理员功能：登录二次元后台管理系统进行各种操作，包括用户管理、作品管理、评论管理、公告管理。

具体功能如图 3-1 所示：

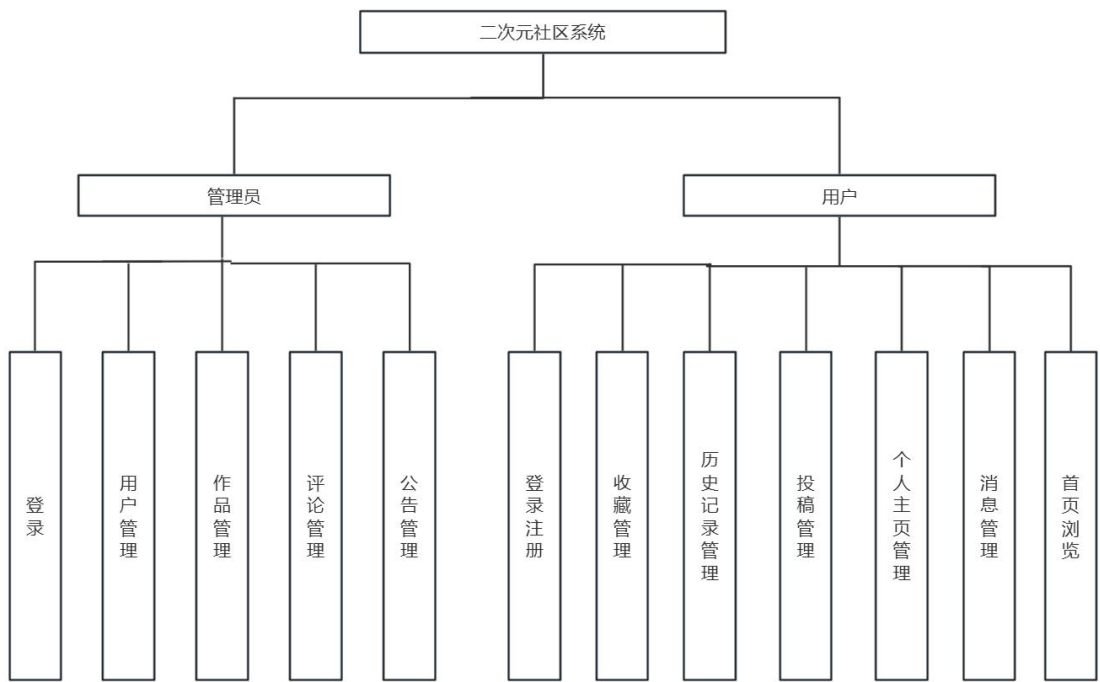


图 3-1 系统功能模块图

3.2 系统功能模块分析

（1）登录功能：登录分为用户登录和管理员登录，用户可以登录或注册进入二次元社区网站，管理员可以登录进入后台管理系统。

（2）用户管理：录入每位用户信息，并保存到数据库中；录入的用户信息包括：用户 id、用户名、昵称、性别、出生年月、个性签名、空间公告及创建时间等属性；并且管理员可以增、删、改用户信息。

（3）作品管理：录入每一个作品信息，并保存到数据库中。作品信息包括：作品 id、作品名、作品封面、作品简介、作者、创建时间、点赞数、收藏数、播

放数、评论数等属性。作品管理分为已审核和未审核两种情况，管理员可以删除作品、修改作品状态。

(4) 评论管理：录入评论信息，并保存到数据库中。评论信息包括：评论 id、评论内容、评论时间、评论人、父级评论、评论作品等属性，管理员可以删除评论。

(5) 公告管理：录入公告信息，并保存到数据库中。公告信息包括：公告 id、公告内容、创建时间等属性，管理员可以增、删、改公告信息。

(6) 首页管理：用户可以在二次元社区网站的首页浏览作品，可以点击任意作品进入详情页查看具体内容，可以点击不同分类查看不同作品。

(7) 收藏管理：用户可以在二次元社区网站的收藏中查看收藏过的作品信息，默认有一个收藏夹，用户可以选择、创建收藏夹、删除、批量删除收藏的作品。

(8) 历史记录管理：用户可以在二次元社区网站的历史记录中查看浏览过的作品信息，用户可以删除、批量删除历史记录。

(9) 个人主页管理：用户可以点击头像进入个人主页、查看自己的个人信息，包括：昵称、性别、个人简介、空间公告、点赞数、收藏数、播放数等信息。同时可以在个人主页查看自己投稿的作品、收藏的作品以及历史记录。

(10) 投稿管理：用户可以点击投稿进入投稿详情页，用户可以自定义投稿作品类型，包括图文投稿和视频投稿。

(11) 消息管理：用户可以点击消息进入消息页，用户可以查看的消息有系统消息、收到的赞、回复我的和评论我的。

总体用例图如图 3-2 所示：

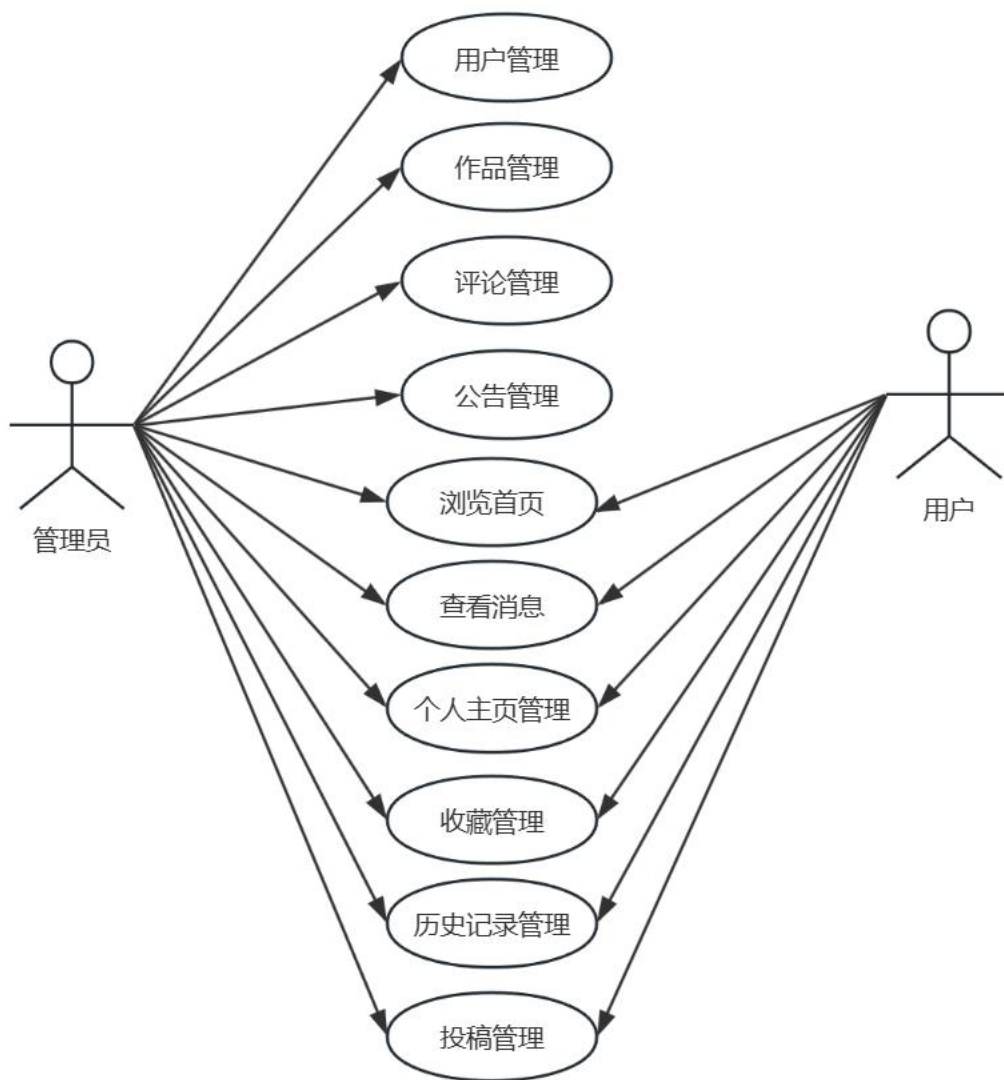


图 3-2 系统总体用例图

4 系统分析与设计

4.1 总体架构设计

二次元社区系统中主要分为四层：

（1）视图层：主要用来显示系统的前台界面。

（2）控制层：主要用于接收客户端请求、处理输入参数、调用业务逻辑，并返回响应。

（3）业务层：主要用于封装业务规则、处理复杂逻辑，并协调多个数据访问操作。

（4）数据层：直接与数据库交互，用于对数据库做一些增删改查的操作，与 service 层配合实现业务逻辑。

二次元社区系统的框架结构如图 4-1 所示。

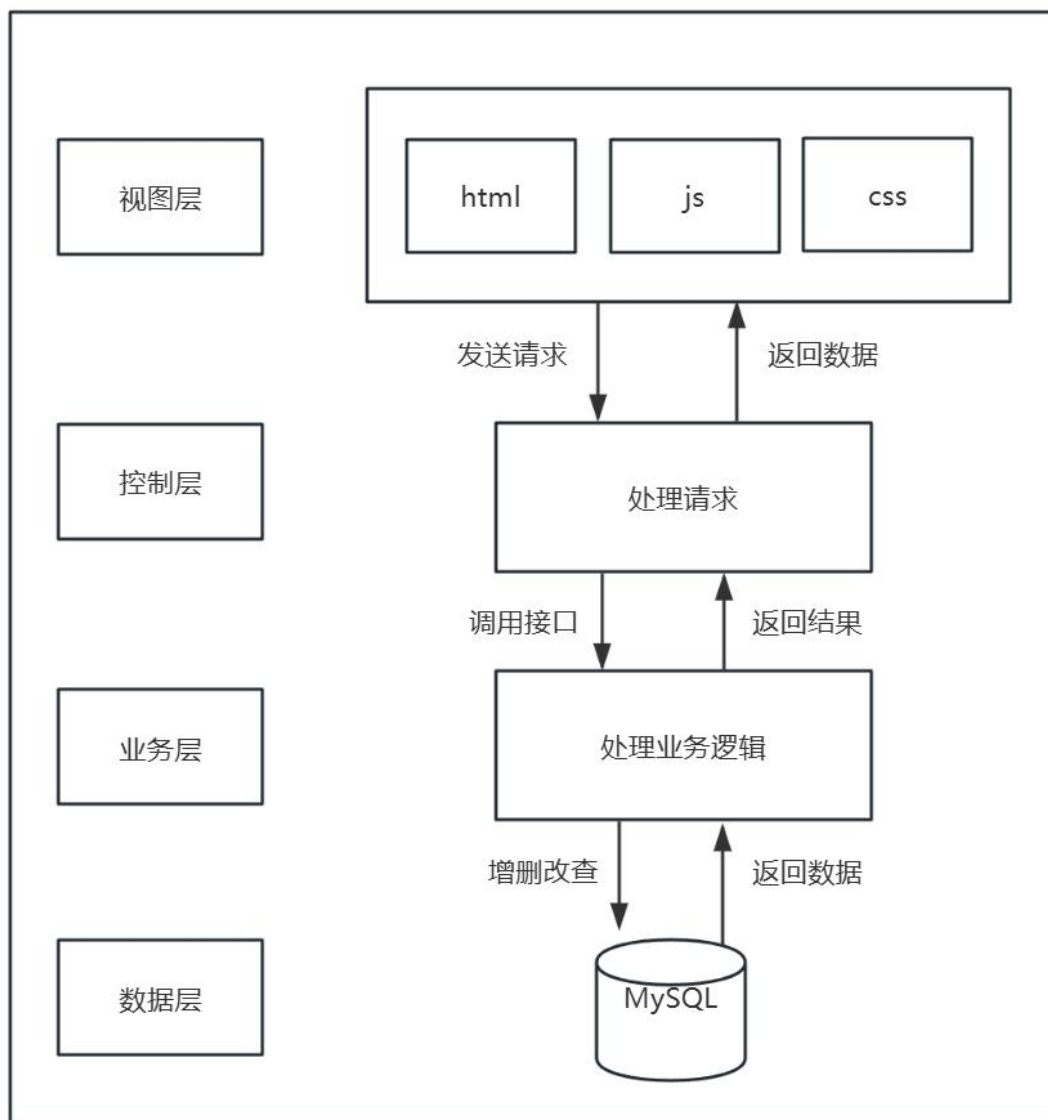


图 4-1 系统总体架构图

4.2 数据库设计

4.2.1 数据库概念模型的设计

概念模型能够清晰地展现数据的逻辑结构。它能够把现实的对象抽象成数据库的这种概念结构，这样可以帮助用户设计出自己的数据库表。通常描述概念模型的方式是绘制 E-R 图将数据转化为规范化的数据表设计方案。

(1) 用户信息实体属性图

用户信息这个实体拥有的属性包括用户 id，用户名，空间公告等属性。用户信息的实体与属性具体如图 4-2 所示。

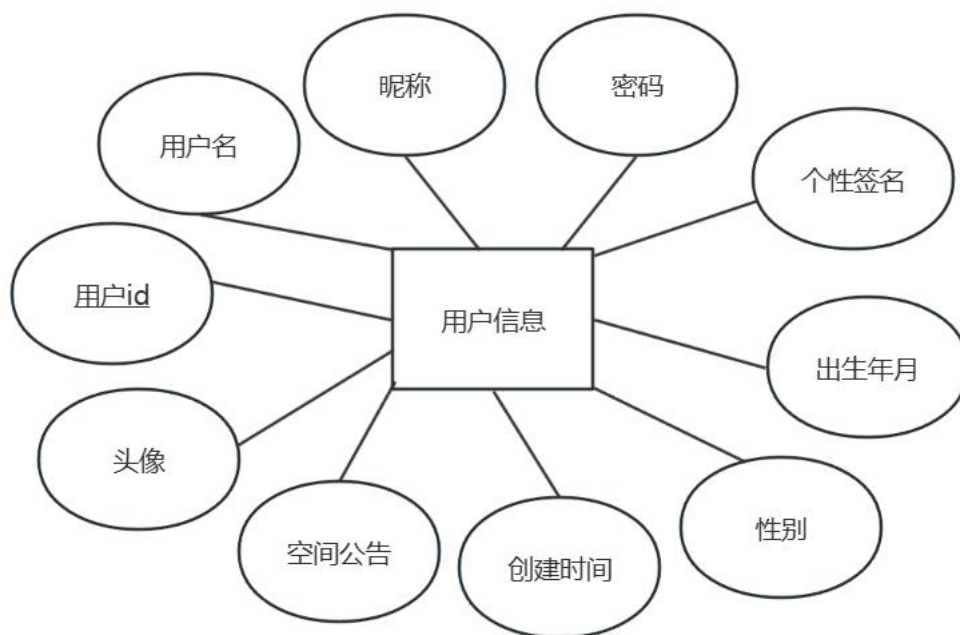


图 4-2 用户信息实体属性图

(2) 作品信息实体属性图

作品信息这个实体拥有的属性包括作品标题，作品简介，作品内容，点赞数，收藏数，播放数，评论数，作品类型等属性。作品信息的实体与属性具体如图 4-3 所示。

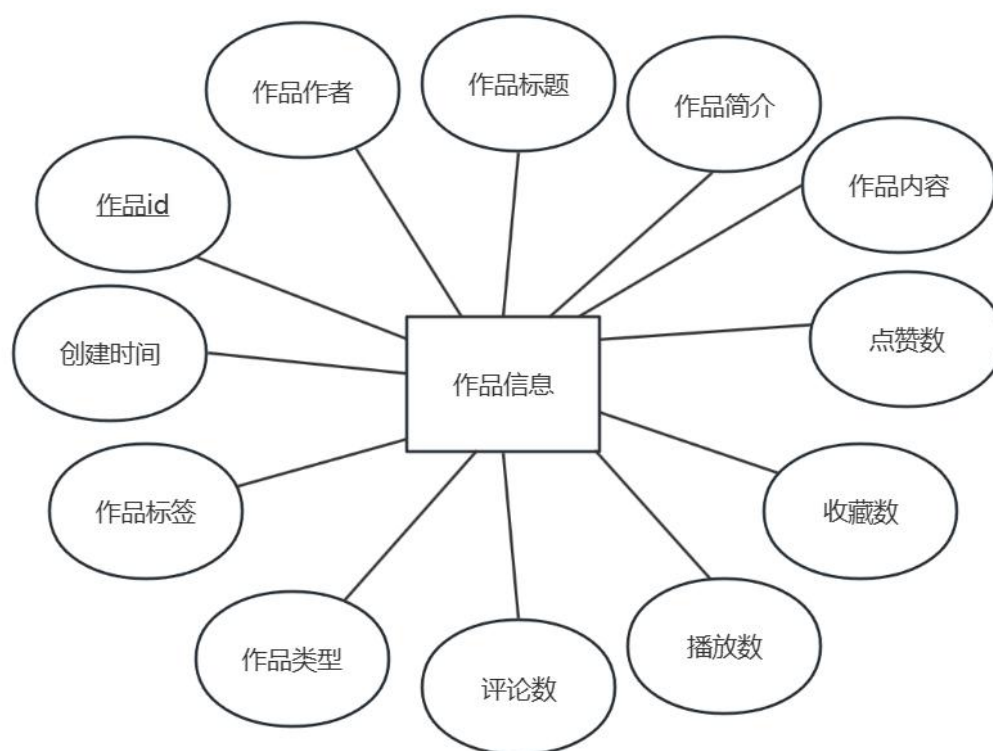


图 4-3 作品信息实体属性图

（3） 二次元 IP 信息实体属性图

二次元 IP 信息这个实体拥有的属性包括 IP 名，IP 图片等属性。二次元 IP 信息的实体与属性具体如图 4-4 所示。

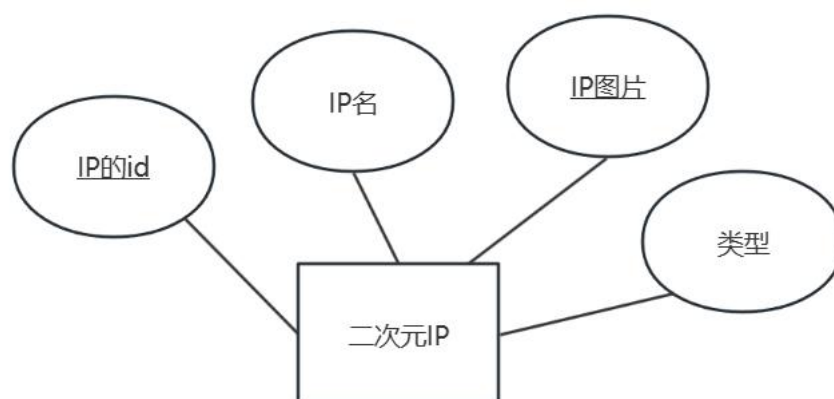


图 4-4 二次元 IP 信息实体属性图

（4） 评论信息实体属性图

评论信息这个实体拥有的属性包括评论内容，父级评论等属性。评论信息的实体与属性具体如图 4-5 所示。

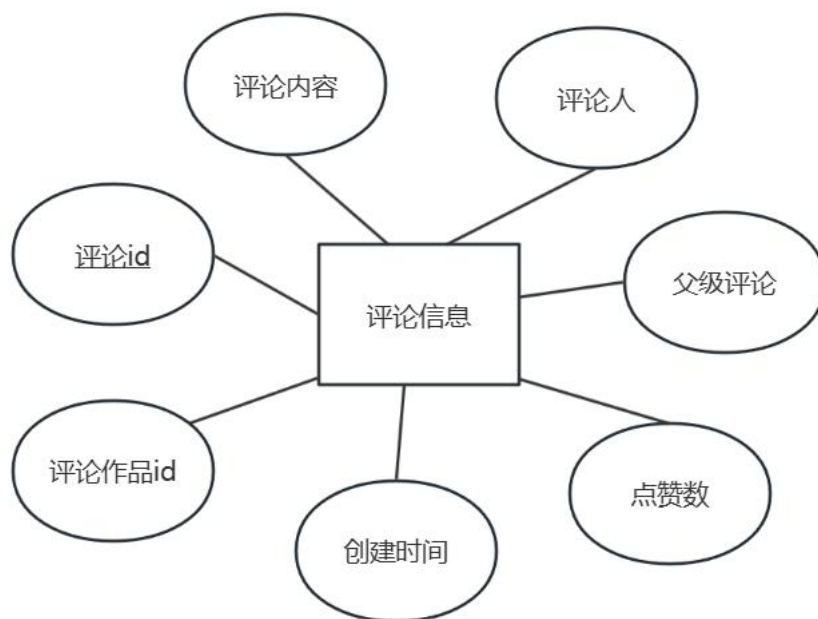


图 4-5 评论信息实体属性图

(5) 公告信息实体属性图

公告信息这个实体拥有的属性包括公告标题等属性。公告信息的实体与属性具体如图 4-6 所示。

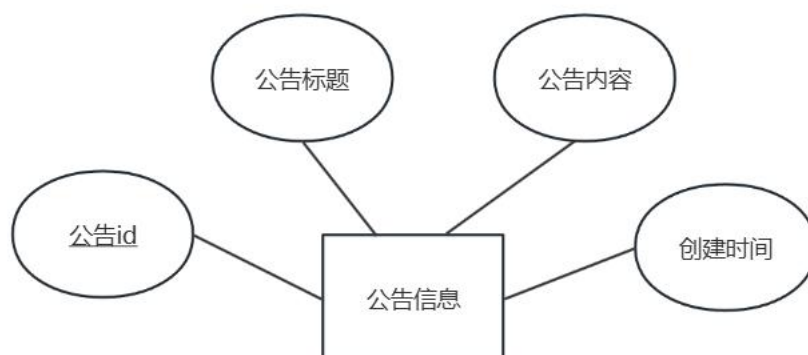


图 4-6 公告信息实体属性图

(6) 作品分类实体属性图

作品分类的实体与属性具体如图 4-7 所示。

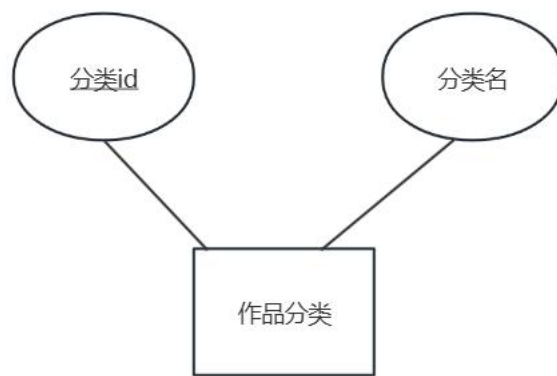


图 4-7 作品分类实体属性图

(7) 作品标签实体属性图

作品标签的实体与属性具体如图 4-8 所示。

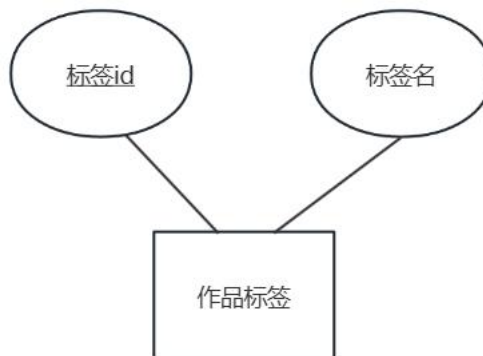


图 4-8 作品标签实体属性图

(8) 收藏实体属性图

收藏这个实体拥有的属性包括收藏人，收藏夹等属性。收藏的实体与属性具体如图 4-9 所示。

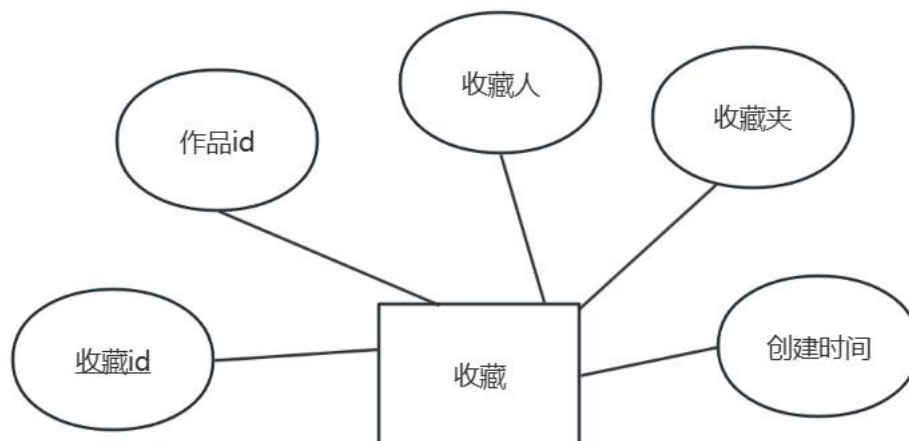


图 4-9 收藏实体属性图

（9）收藏夹实体属性图

收藏夹这个实体拥有的属性包括收藏夹名称，收藏夹的所属人等属性。收藏夹的实体与属性具体如图 4-10 所示。

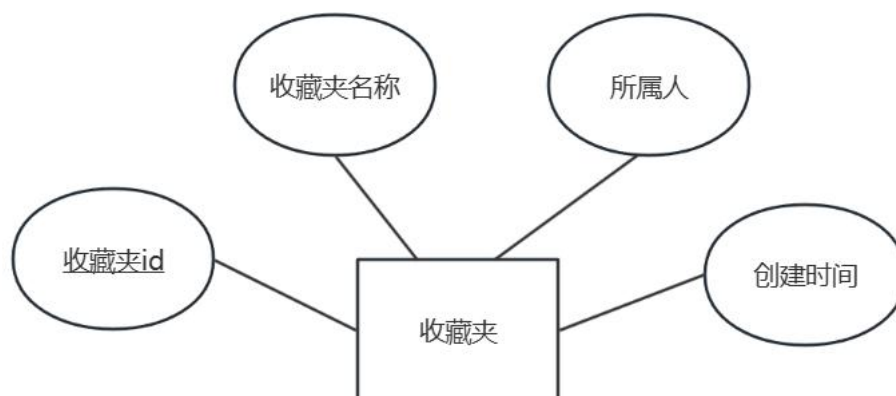


图 4-10 收藏夹实体属性图

（10）历史记录实体属性图

历史记录这个实体拥有的属性包括历史作品，观看人等属性。历史记录的实体与属性具体如图 4-11 所示。

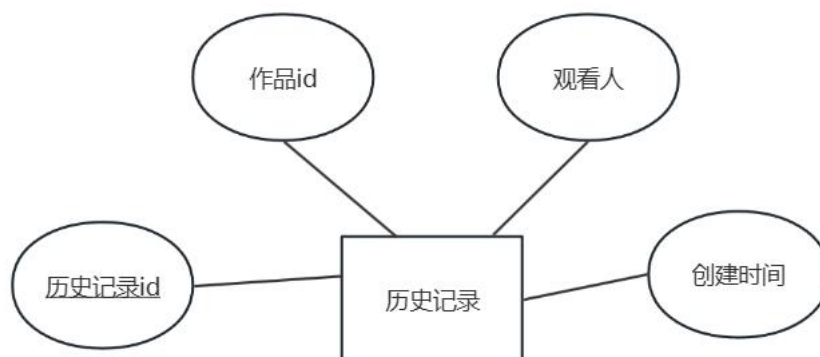


图 4-11 历史记录实体属性图

（11）系统整体 E-R 图

针对于二次元社区系统的整体 E-R 图，对其具体的关系逻辑进行分析：一个用户可以发表多个评论，一条评论只属于一个用户，所以用户与评论之间是 1:n 的关系；一个用户可以投稿多个作品，一个作品只属于一个用户，所以用户与作品之间是 1:n 的关系；一个用户可以查看多个公告，一个公告可以发送给多个用户，所以用户与公告之间是 n:m 的关系；一个用户可以拥有多条历史记录，一条历史记录只能属于一个用户，所以用户与历史记录之间是 1:n 的关系；一个用户

可以拥有多个收藏夹，一个收藏夹只能属于一个用户，所以用户与收藏夹之间是 1:n 的关系；一个收藏夹可以拥有多个收藏作品，一个收藏作品只能存在于一个收藏夹，所以收藏夹与收藏之间是 1:n 的关系；一个作品可以包含多个作品标签，一个作品标签可以被多个作品使用，所以作品与作品标签之间是 n:m 的关系；一个作品只能属于一个作品分类，一个作品分类可以包含多个作品，所以作品与作品分类之间是 n:1 的关系；一个作品只能属于一个二次元 IP，一个二次元 IP 可以包含多个作品，所以作品与二次元 IP 之间是 n:1 的关系。接下来通过一个整体的 E-R 图展示，如图 4-12 所示。

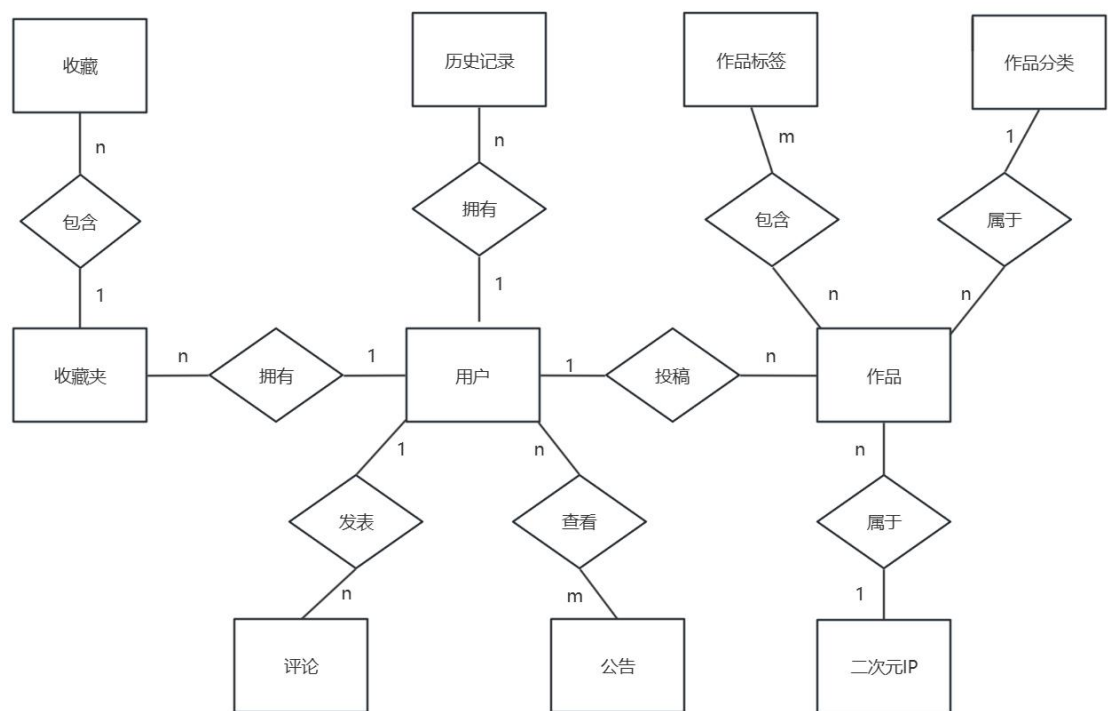


图 4-12 系统整体 E-R 图

4.2.2 概念模型转为关系模型

为了将上述数据库概念模型转换为数据库能够支持的关系模型，需要完整的列出实体和属性以及实体之间的逻辑关系，使其能够映射到特定的数据库逻辑结构中。再基于实体和实体之间的关系，形成数据库中表之间的关系和每个表的属性之间的关系。

下面展示实体及其属性：

用户信息（用户 ID，用户名，头像，个性签名，性别，密码，创建时间，空间公告）

作品信息（作品 ID，作品简介，作品类型，作品标签，作品内容，作品作者，作品标题，点赞数，收藏数，播放数，评论数，创建时间）

二次元 IP 信息（IP 的 ID，IP 名，IP 图片，类型，创建时间）

评论信息（评论 ID，评论内容，评论人，评论作品，父级评论，点赞数，创建时间）

公告信息（公告 ID，公告标题，公告内容，创建时间）

历史记录（历史记录 ID，作品 ID，观看人，创建时间）

收藏（收藏 ID，作品 ID，收藏人，收藏夹，创建时间）

收藏夹（收藏夹 ID，所属人，收藏夹名称，创建时间）

作品标签（标签 ID，标签名）

作品分类（分类 ID，分类名）

4.2.3 数据库表的设计

（1）用户信息表

表 4-1 用户信息表

字段	类型	说明
user_id	bigint(20)	用户 ID 主键
user_name	varchar(20)	用户名（唯一）
sex	char(1)	性别（0 女 1 男）
nick_name	varchar(20)	昵称
password	varchar(50)	密码
avatar	varchar(200)	头像地址
birthday	date	出生年月
personal_introduction	varchar(200)	个性签名
notice_info	varchar(50)	个人主页展示的空间公告
create_time	datetime	创建时间

（2）作品信息表

表 4-2 作品信息表

字段	类型	说明
work_id	bigint(20)	作品 ID 主键
work_user	bigint(20)	作品作者 ID
work_title	varchar(100)	作品标题
work_introduction	text	作品简介
content_path	varchar(200)	作品内容的路径
like_count	int	作品点赞数
collect_count	int	作品收藏数
play_count	int	作品播放数
comment_count	int	作品评论数
work_type	varchar(10)	作品类型
work_tag	varchar(50)	作品标签
create_time	datetime	创建时间

(3) 二次元 IP 信息表

表 4-3 二次元 IP 信息表

字段	类型	说明
ip_id	bigint(10)	二次元 IP 的 ID 主键
ip_name	varchar(20)	二次元 IP 名
ip_type	varchar(20)	二次元 IP 所属类型
ip_img	varchar(20)	二次元 IP 图片
create_time	datetime	创建时间

(4) 评论信息表

表 4-4 评论信息表

字段	类型	说明
comment_id	bigint(20)	评论 ID 主键
comment_text	text	评论内容
comment_people	bigint(10)	评论人 ID
parent_id	bigint(20)	父级评论 ID
like_count	int	点赞数
comment_work	date	评论的作品 ID
create_time	datetime	创建时间

(5) 公告信息表

表 4-5 公告信息表

字段	类型	说明
notice_id	bigint(10)	公告 ID 主键
notice_title	varchar(100)	公告标题
notice_text	text	公告内容
create_time	datetime	创建时间

(6) 作品分类表

表 4-6 作品分类表

字段	类型	说明
class_id	bigint(20)	分类 ID 主键
class_name	varchar(50)	分类名称

(7) 作品标签表

表 4-7 作品标签表

字段	类型	说明
tag_id	bigint(20)	标签 ID 主键
tag_name	varchar(20)	标签名称

(8) 收藏夹表

表 4-8 收藏夹表

字段	类型	说明
favorites_id	bigint(20)	收藏夹 ID 主键
favorites_people	bigint(20)	收藏夹所属用户 ID
favorites_name	varchar(10)	收藏夹名称

create_time	datetime	创建时间
-------------	----------	------

(9) 收藏表

表 4-9 收藏表

字段	类型	说明
collect_id	bigint(20)	收藏 ID 主键
collect_people	bigint(20)	收藏人 ID
collect_list	bigint(20)	收藏夹 ID
collect_work	bigint(20)	收藏的作品 ID
create_time	datetime	创建时间

(10) 历史记录表

表 4-10 历史记录表

字段	类型	说明
history_id	bigint(20)	历史记录 ID 主键
history_work	bigint(20)	观看过的作品 ID
history_people	bigint(20)	历史记录所属人 ID
create_time	datetime	创建时间

5 系统的开发设计与实现

5.1 登录模块

5.1.1 登录模块界面设计

用户进入二次元社区，默认情况下可以浏览社区作品，但无法进行个人行为，如查看消息、点赞、收藏等行为，可以通过点击用户头像或者进行个人行为操作时弹出登录弹窗，如图 5-1 所示。

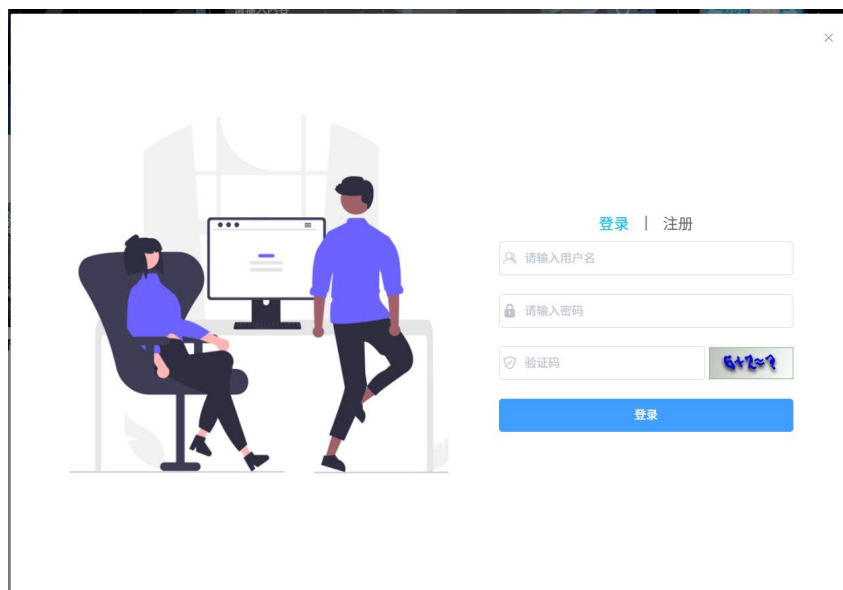


图 5-1 用户登录界面设计图

管理员进入二次元社区后台管理系统前，默认先进入登录页，如图 5-2 所示。

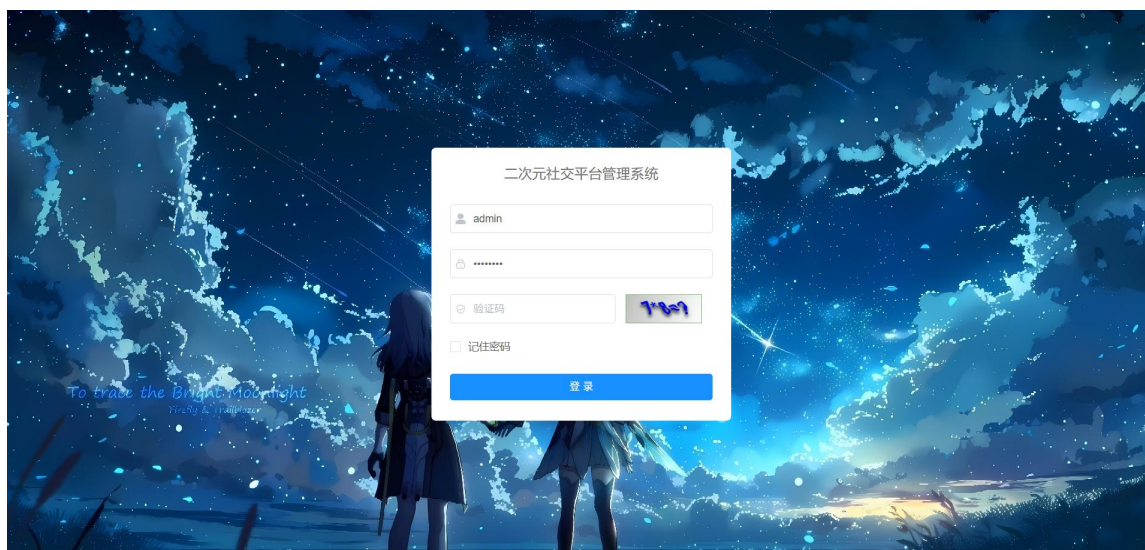


图 5-2 管理员登录界面设计图

5.1.2 登录模块功能设计

用户在登录时，需要输入用户名、密码和验证码，点击确认后，系统先检测

验证码是否正确，若不正确，提示用户重新输入验证码，然后系统会检查用户表里是否有此用户，若无此用户，系统会弹出提示框，提示用户去进行注册，若有此用户，系统会检测用户名或密码是否正确，若不正确，系统会提示用户重新输入用户名或密码，最后都检测通过后，用户即可体验个性化二次元社区。具体流程图如图 5-3 所示。

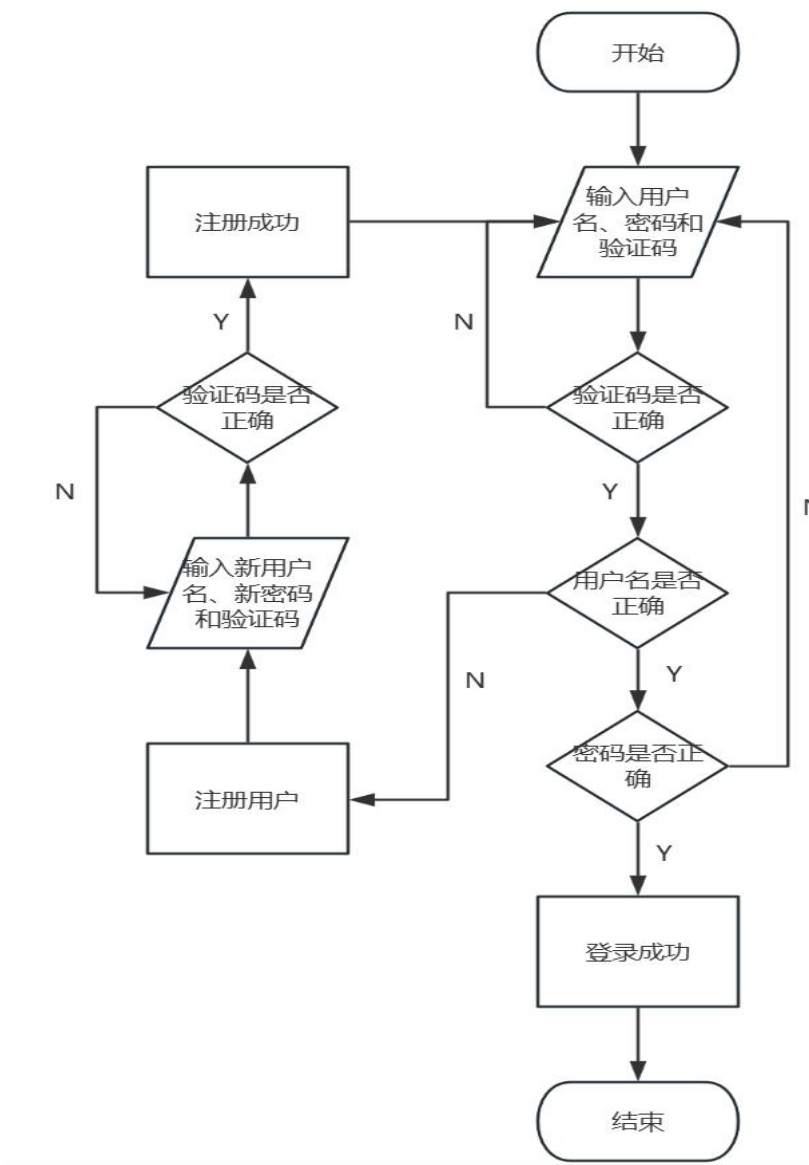


图 5-3 用户登录流程图

管理员在登录时，需要输入用户名、密码和验证码，点击确认后，系统先检测验证码是否正确，若不正确，提示管理员重新输入验证码，然后系统会检测用户名或密码是否正确，若不正确，系统会提示管理员重新输入用户名或密码，最后都检测通过后，管理员即可进入二次元社区后台管理系统。具体流程图如图 5-4 所示。

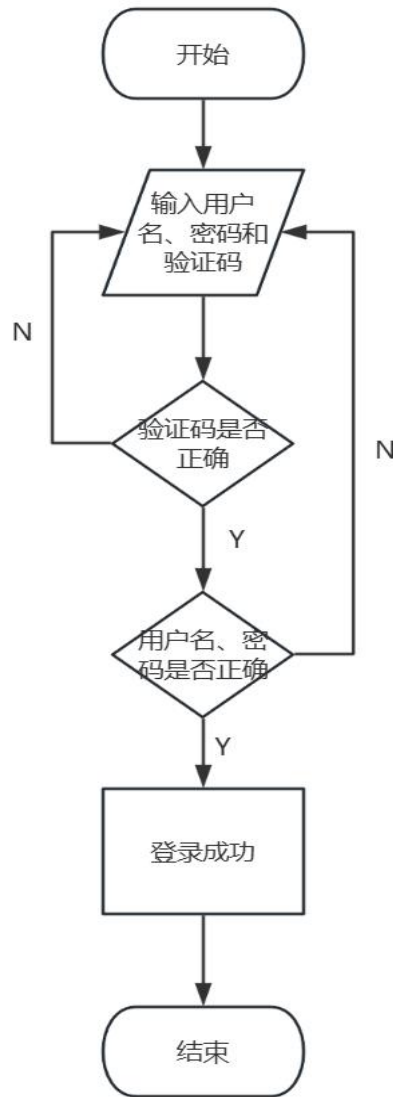


图 5-4 管理员登录流程图

5.1.3 登录模块核心代码

登录模块主要需要实现验证码的验证功能以及用户名和密码的验证功能，以下是该功能的核心代码：

```

@Service
public class LoginServiceImpl implements LoginService {
    @Autowired
    private UserMapper userMapper;
    @Autowired
    private PasswordEncoder passwordEncoder;
    @Override
    public boolean verifyCaptcha(String inputCaptcha, HttpSession session) {
        String sessionCaptcha = (String) session.getAttribute("CAPTCHA");
        session.removeAttribute("CAPTCHA");
        return inputCaptcha.equalsIgnoreCase(sessionCaptcha);
    }
}
  
```

```

@Override
public User login(String username, String password) {
    User user = userMapper.selectByUsername(username);
    if (user == null) {
        throw new BusinessException("用户不存在，请注册");
    }
    if (!passwordEncoder.matches(password, user.getPassword())) {
        throw new BusinessException("用户名或密码错误");
    }
    if (user.getStatus() != UserStatus.NORMAL) {
        throw new BusinessException("账号已被禁用");
    }
    return user;
}

@Override
public String generateToken(User user) {
    return JwtUtil.generateToken(user.getId(), user.getRole());
}
}

```

5.2 用户管理模块

5.2.1 用户管理模块界面设计

二次元社区后台管理系统的用户管理模块界面设计如图 5-5 所示，管理员可以查看用户信息，删除用户，重置用户密码，根据用户昵称搜索用户，批量选择用户然后进行删除用户或重置用户密码操作。

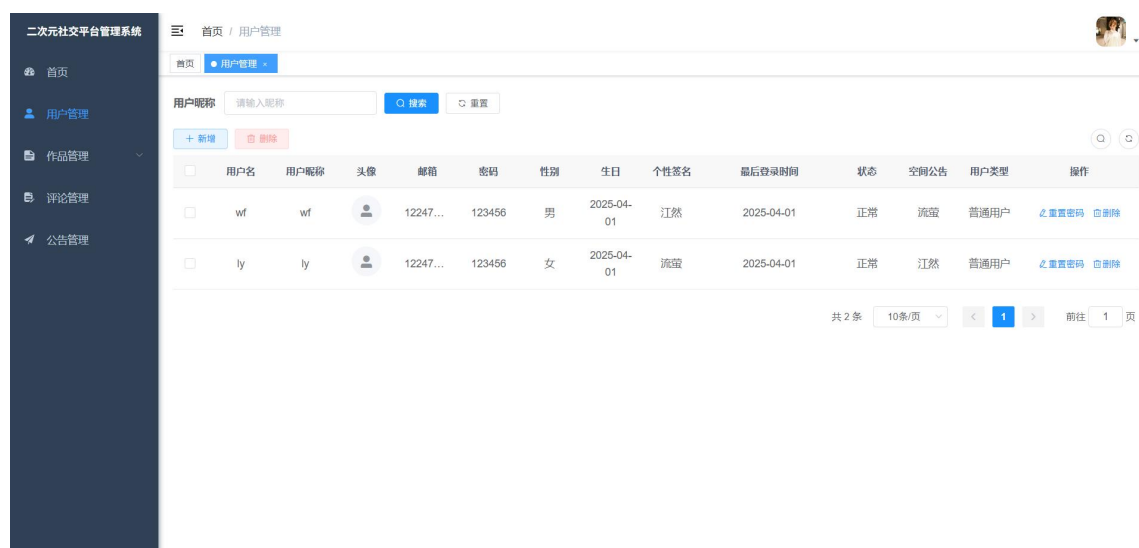


图 5-5 用户管理界面设计图

5.2.2 用户管理模块功能设计

管理员进入到系统后，对于用户管理模块进行最频繁的操作是批量选择用户，

然后进行删除或重置密码操作。流程图如图 5-6 所示。

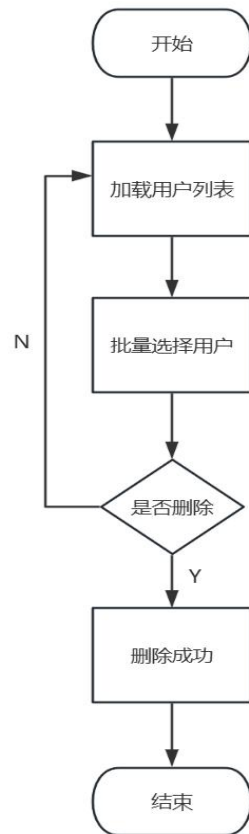


图 5-6 批量删除用户流程图

5.2.3 用户管理模块核心代码

用户管理主要需要实现对用户表的一系列修改和删除操作，以下是该功能的核心代码：

```
@Service
public class UserAdminServiceImpl implements UserAdminService {
    @Autowired
    private UserMapper userMapper;
    @Autowired
    private PasswordEncoder passwordEncoder;
    @Autowired
    private AuditLogService auditLogService;
    @Override
    public PageResult<UserVO> getUsers(PageParam pageParam, UserQueryDTO query) {
        PageHelper.startPage(pageParam);
        List<User> users = userMapper.selectByCondition(query);
        return new PageResult<>(users.stream()
            .map(this::convertToVO)
            .collect(Collectors.toList()));
    }
    private UserVO convertToVO(User user) {
```

```

        return new UserVO()
            .setId(user.getId())
            .setUsername(user.getUsername())
            .setAvatar(user.getAvatar())
            .setStatus(user.getStatus());
    }
}

```

5.3 作品管理模块

5.3.1 作品管理模块界面设计

二次元社区后台管理系统的作品管理模块界面设计如图 5-7 所示，管理员可以查看用户投稿的作品信息，根据作品标题搜索作品，批量选择作品然后进行操作，其中作品分为已通过和未通过两种情况，管理员对于未通过的作品可以进行审核，对于已通过的作品可以查看详情和驳回作品的审核通过状态。



图 5-7 作品管理界面设计图

5.3.2 作品管理模块功能设计

管理员在作品管理的未通过页面审核作品会先进入作品详情，然后可以选择是否审核通过。流程图如图 5-8 所示。

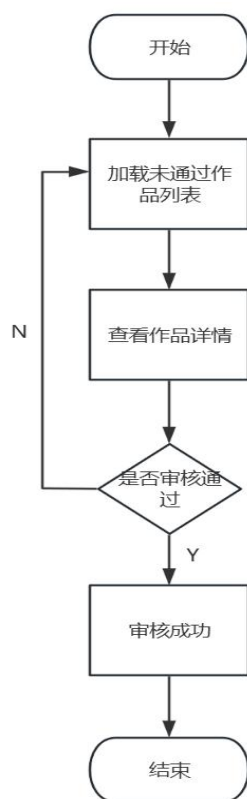


图 5-8 作品审核流程图

5.3.3 作品管理模块核心代码

作品管理主要是实现对作品表的一系列增删改查操作，以下是该功能的核心代码：

@Service

```

public class WorkServiceImpl implements WorkService {
    @Autowired
    private WorkMapper workMapper;
    @Autowired
    private TagService tagService;
    @Autowired
    private ContentCheckService contentCheck;
    @Override
    public Long publishWork(WorkPublishDTO dto, Long authorId) {
        if (!contentCheck.checkText(dto.getContent())) {
            throw new BusinessException("内容包含违规信息");
        }
        Work work = new Work()
            .setAuthorId(authorId)
            .setTitle(dto.getTitle())
            .setContent(dto.getContent())
            .setCoverImage(dto.getCoverImage())
            .setStatus(WorkStatus.PENDING);
        workMapper.insert(work);
    }
}
  
```

```

        tagService.bindTags(work.getId(), dto.getTags());
        auditService.submitForReview(work.getId());
        return work.getId();
    }

    @Override
    public void batchReview(List<Long> workIds, ReviewStatus status, String reason) {
        if (status == ReviewStatus.APPROVED && StringUtils.isBlank(reason)) {
            throw new BusinessException("通过审核必须填写备注");
        }
        workMapper.batchUpdateStatus(workIds, status);
        auditLogService.logReviewAction(
            SecurityUtil.getCurrentUserId(),
            workIds,
            status,
            reason
        );
        if (status == ReviewStatus.APPROVED) {
            recommendationService.processNewWorks(workIds);
        }
    }

    @Override
    public PageResult<WorkVO> queryWorks(WorkQueryDTO query, PageParam pageParam) {
        PageHelper.startPage(pageParam);
        List<Work> works = workMapper.selectByCondition(query);
        return convertToPageResult(works);
    }
}

```

5.4 评论管理模块

5.4.1 评论管理模块界面设计

二次元社区后台管理系统的评论管理模块界面设计如图 5-9 所示，管理员可以查看用户的评论信息，删除评论，根据评论内容搜索评论，批量选择评论然后进行删除操作。

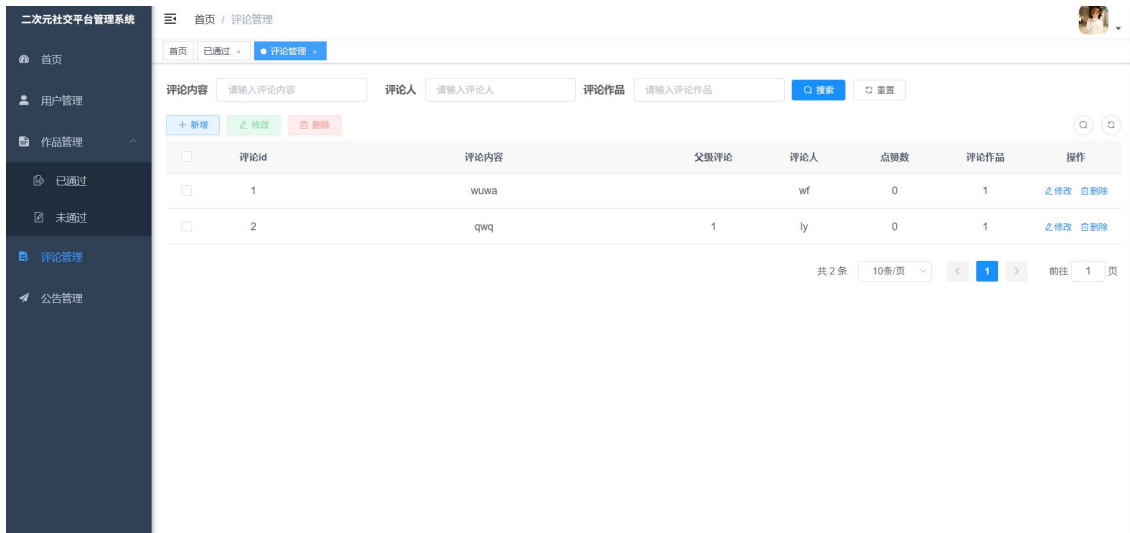


图 5-9 评论管理界面设计图

5.4.2 评论管理模块功能设计

管理员在评论管理模块，搜索包含指定评论内容的全部评论，然后批量选择后删除评论。流程图如图 5-10 所示。

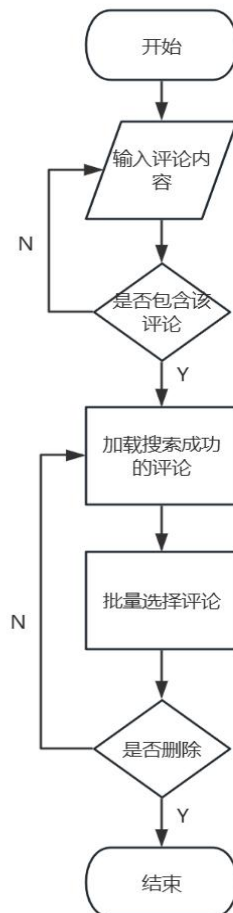


图 5-10 批量删除搜索的评论流程图

5.4.3 评论管理模块核心代码

评论管理主要需要实现对评论表的一系列增删改查操作，以下是该功能的核心

代码:

@Service

```
public class CommentService {  
    @Autowired  
    private CommentRepository commentRepository;  
    public Comment addComment(Comment comment) {  
        comment.setStatus("pending");  
        comment.setCreatedAt(LocalDateTime.now());  
        return commentRepository.save(comment);  
    }  
    public void deleteComment(Long id) {  
        commentRepository.deleteById(id);  
    }  
    public Comment updateComment(Long id, String newContent) {  
        Optional<Comment> optionalComment = commentRepository.findById(id);  
        if (optionalComment.isPresent()) {  
            Comment comment = optionalComment.get();  
            comment.setContent(newContent);  
            comment.setUpdatedAt(LocalDateTime.now());  
            return commentRepository.save(comment);  
        }  
        throw new RuntimeException("Comment not found with ID: " + id);  
    }  
    public Page<Comment> getComments(int page, int size, String status) {  
        Pageable pageable = PageRequest.of(page, size);  
        if (status != null && !status.isEmpty()) {  
            return commentRepository.findByStatus(status, pageable);  
        }  
        return commentRepository.findAll(pageable);  
    }  
}
```

5.5 公告管理模块

5.5.1 公告管理模块界面设计

在二次元社区后台管理系统的公告管理模块，管理员可以添加公告，删除公告，查看系统的公告信息，根据公告内容搜索公告，批量选择公告然后进行删除操作，界面设计如图 5-11 所示。

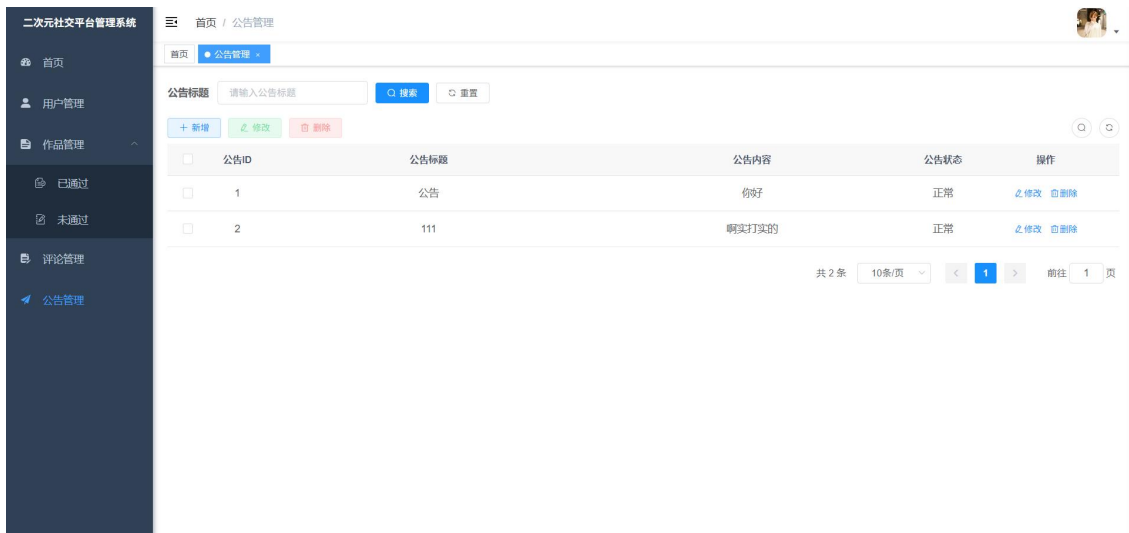


图 5-11 公告管理界面设计图

5.5.2 公告管理模块功能设计

管理员在公告管理模块，点击添加按钮，输入公告信息，系统检测填入的信息是否符合要求，若符合则添加成功，若不符合则重新输入公告信息。流程图如图 5-12 所示。

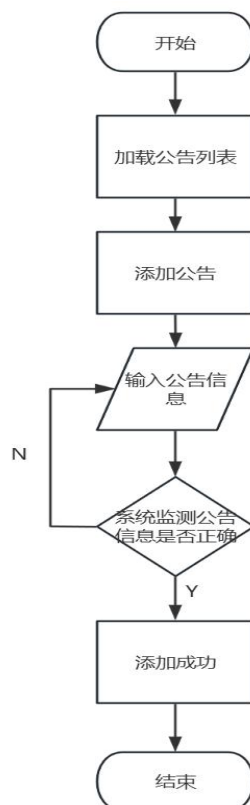


图 5-12 添加公告流程图

5.5.3 公告管理模块核心代码

公告管理主要需要实现对公告表的一系列增删改查操作，以下是该功能的核心代码：

```

@Service
public class AnnouncementService {
    @Autowired
    private AnnouncementRepository announcementRepository;
    public Announcement addAnnouncement(Announcement announcement) {
        announcement.setStatus("draft");
        announcement.setCreatedAt(LocalDateTime.now());
        return announcementRepository.save(announcement);
    }
    public void deleteAnnouncement(Long id) {
        announcementRepository.deleteById(id);
    }
    public Announcement updateAnnouncement(Long id, Announcement updatedAnnouncement) {
        Optional<Announcement> optionalAnnouncement =
announcementRepository.findById(id);
        if (optionalAnnouncement.isPresent()) {
            Announcement announcement = optionalAnnouncement.get();
            announcement.setTitle(updatedAnnouncement.getTitle());
            announcement.setContent(updatedAnnouncement.getContent());
            announcement.setUpdatedAt(LocalDateTime.now());
            return announcementRepository.save(announcement);
        }
        throw new RuntimeException("Announcement not found with ID: " + id);
    }
    public Page<Announcement> getAnnouncements(int page, int size, String status) {
        Pageable pageable = PageRequest.of(page, size);
        if (status != null && !status.isEmpty()) {
            return announcementRepository.findByStatus(status, pageable);
        }
        return announcementRepository.findAll(pageable);
    }
    public Announcement publishAnnouncement(Long id) {
        Optional<Announcement> optionalAnnouncement =
announcementRepository.findById(id);
        if (optionalAnnouncement.isPresent()) {
            Announcement announcement = optionalAnnouncement.get();
            announcement.setStatus("published");
            announcement.setPublishedAt(LocalDateTime.now());
            return announcementRepository.save(announcement);
        }
        throw new RuntimeException("Announcement not found with ID: " + id);
    }
    public Announcement unpublishAnnouncement(Long id) {
        Optional<Announcement> optionalAnnouncement =
announcementRepository.findById(id);

```

```

        if (optionalAnnouncement.isPresent()) {
            Announcement announcement = optionalAnnouncement.get();
            announcement.setStatus("draft");
            return announcementRepository.save(announcement);
        }
        throw new RuntimeException("Announcement not found with ID: " + id);
    }
}

```

5.6 首页模块

5.6.1 首页模块界面设计

用户在登录成功后进入首页，如图 5-13 所示。在首页中会展示整个社区的热门评论，以弹幕雨的形式。首页模块的所有用户相关功能都可以供用户使用，可以点击任意一个作品进入作品详情页查看作品信息，同时用户也可以在首页进入用户自己的个人主页、消息页、收藏页、历史记录页以及投稿页面。

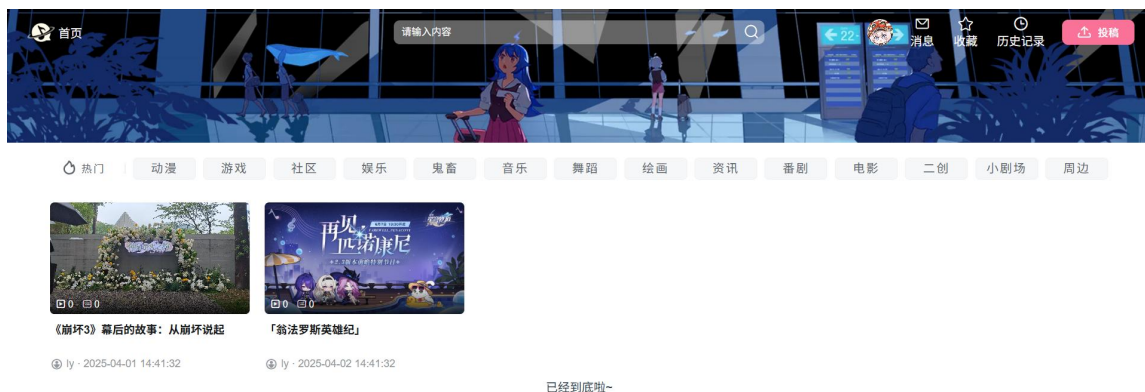


图 5-13 首页界面设计图

5.6.2 首页模块功能设计

用户进入首页后，可以点击任意一个作品，系统检测该作品的作品 id，然后进入该作品的详情页。流程图如图 5-14 所示。



图 5-14 查看作品详情流程图

5.6.3 首页模块核心代码

首页模块主要需要对作品表的一系列增删改查操作去控制显示首页的作品，以下是该功能的核心代码：

```

@Service
public class MysWorkPassInfoServiceImpl implements IMysWorkPassInfoService
{
    @Autowired
    private MysWorkPassInfoMapper mysWorkPassInfoMapper;

    @Override
    public MysWorkPassInfo selectMysWorkPassInfoByWorkId(Long workId)
    {
        return mysWorkPassInfoMapper.selectMysWorkPassInfoByWorkId(workId);
    }

    @Override
    public List<MysWorkPassInfo> selectMysWorkList()
    {
        return mysWorkPassInfoMapper.selectMysWorkList();
    }

    @Override
    public List<MysWorkPassInfo> selectMysWorkPassInfoList(MysWorkPassInfo
mysWorkPassInfo)

```

```

    {
        return mysWorkPassInfoMapper.selectMysWorkPassInfoList(mysWorkPassInfo);
    }
}

```

5.7 消息模块

5.7.1 消息模块界面设计

用户在二次元社区首页点击消息，可以进入自己的消息页，如图 5-15 所示，消息页一共有 4 个部分，分别是回复我的、评论我的、收到的赞和系统消息。



图 5-15 消息页界面设计图

5.7.2 消息模块功能设计

用户进入消息页，点击系统消息，显示当前用户收到的系统消息，点击其他部分也是同理。流程图如图 5-16 所示。



图 5-16 查看系统消息流程图

5.7.3 消息模块核心代码

消息模块主要需要实现对消息表和评论表的一系列增删改查操作，以下是该功能的核心代码：

```

@Override
public MysNotice selectMysNoticeByNoticeId(Integer noticeId)
{
    return mysNoticeMapper.selectMysNoticeByNoticeId(noticeId);
}

@Override
public MysCommentInfo selectMysCommentInfoByCommentId(Long commentId)
{
    return mysCommentInfoMapper.selectMysCommentInfoByCommentId(commentId);
}
  
```

5.8 收藏模块

5.8.1 收藏模块界面设计

用户在二次元社区首页点击收藏，可以进入自己的收藏页，如图 5-17 所示，默认显示全部收藏的作品信息，并放在默认收藏夹。用户可以添加收藏夹，查看自己所添加的收藏夹的作品信息。用户可以取消收藏，可以将指定收藏作品移动到另一个收藏夹。



图 5-17 收藏页界面设计图

5.8.2 收藏模块功能设计

用户进入收藏页面，点击添加收藏夹，系统检测输入的收藏夹信息是否正确，成功后将显示新添加的收藏夹。流程图如图 5-18 所示。

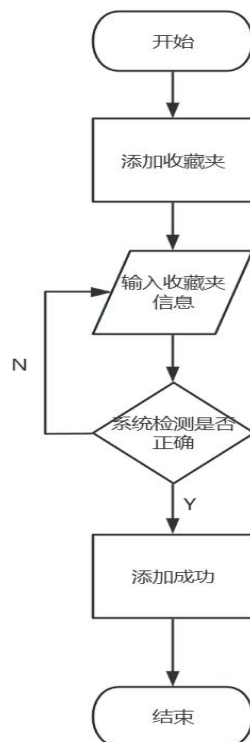


图 5-18 添加收藏夹流程图

5.8.3 收藏模块核心代码

收藏模块主要需要实现对收藏表的一系列增删改查操作以及与收藏夹表的关联，以下是该功能的核心代码：

```

@Service
public class FavoriteService {
    @Autowired
    private FavoriteRepository favoriteRepository;
    public Favorite addFavorite(Long userId, Long contentId, String contentType) {

```

```

        Optional<Favorite> optionalFavorite =
favoriteRepository.findByUserIdAndContentIdAndContentType(userId, contentId, contentType);
        if (optionalFavorite.isPresent()) {
            throw new RuntimeException("Already favorited");
        }
        Favorite favorite = new Favorite();
        favorite.setUserId(userId);
        favorite.setContentId(contentId);
        favorite.setContentType(contentType);
        favorite.setCreatedAt(LocalDateTime.now());
        return favoriteRepository.save(favorite);
    }
    public void removeFavorite(Long userId, Long contentId, String contentType) {
        Optional<Favorite> optionalFavorite =
favoriteRepository.findByUserIdAndContentIdAndContentType(userId, contentId, contentType);
        if (optionalFavorite.isPresent()) {
            favoriteRepository.delete(optionalFavorite.get());
        } else {
            throw new RuntimeException("Favorite not found");
        }
    }
    public Page<Favorite> getUserFavorites(Long userId, int page, int size) {
        Pageable pageable = PageRequest.of(page, size);
        return favoriteRepository.findByUserId(userId, pageable);
    }
    public boolean isFavorited(Long userId, Long contentId, String contentType) {
        return favoriteRepository.findByUserIdAndContentIdAndContentType(userId, contentId,
contentType).isPresent();
    }
}

```

5.9 历史记录模块

5.9.1 历史记录模块界面设计

用户在二次元社区首页点击历史记录，可以进入自己的历史记录页，如图 5-19 所示，默认显示用户已浏览过的作品信息。用户可以删除任意一条历史记录，也可以选择一键删除所有的历史记录。



图 5-19 历史记录页界面设计图

5.9.2 历史记录模块功能设计

用户进入历史记录页，选择指定作品点击删除后，确认是否删除，确认后则会直接删除该作品从历史记录页。流程图如图 5-20 所示。

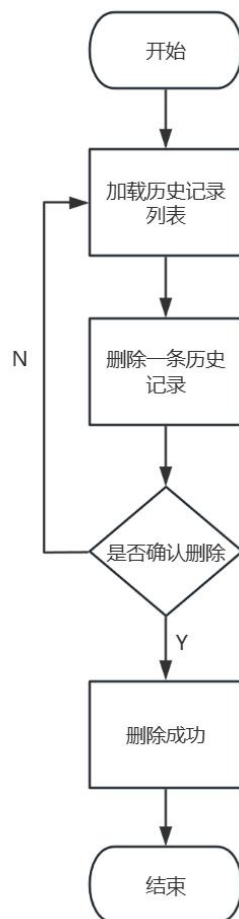


图 5-20 删除历史记录流程图

5.9.3 历史记录模块核心代码

历史记录模块主要需要实现对历史记录表的一系列增删改查操作，以下是该功能模块的核心代码：

```

@Service
public class HistoryService {
    @Autowired
    private HistoryRepository historyRepository;
    public History addHistory(Long userId, String actionType, String contentId, String details) {
        History history = new History();
        history.setUserId(userId);
        history.setActionType(actionType);
        history.setContentId(contentId);
        history.setDetails(details);
        history.setCreatedAt(LocalDateTime.now());
        return historyRepository.save(history);
    }
    public Page<History> getUserHistories(Long userId, int page, int size) {
        Pageable pageable = PageRequest.of(page, size);
        return historyRepository.findByUserId(userId, pageable);
    }
    public void clearUserHistories(Long userId) {
        historyRepository.deleteByUserId(userId);
    }
}

```

5.10 投稿管理模块

5.10.1 投稿管理模块界面设计

用户在二次元社区首页点击投稿，可以进入投稿管理页，如图 5-21 所示，分为视频投稿和图文投稿。用户可以选择一种投稿方式，将自己的作品投稿到系统，审核通过后即可在首页随机刷到自己的作品，也可在个人主页查看自己投稿的作品。

图 5-21 作品投稿界面设计图

5.10.2 投稿管理模块功能设计

用户进入到投稿管理页，选择任意一种投稿方式，输入投稿所需要的作品信息，确认后经过系统审核，审核通过即为投稿成功，反之需要重新投稿。流程图如图 5-22 所示。

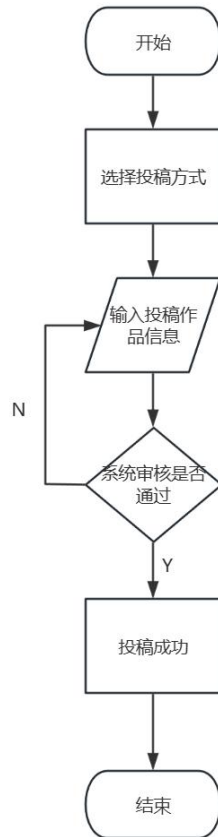


图 5-22 投稿作品流程图

5.10.3 投稿管理模块核心代码

投稿管理模块主要需要考虑用户投稿作品以及管理员审核作品的逻辑代码，以下是该功能的核心代码：

```
@Service
public class SubmissionService {
    @Autowired
    private SubmissionRepository submissionRepository;
    public Submission addSubmission(Submission submission) {
        submission.setStatus("pending");
        submission.setCreatedAt(LocalDateTime.now());
        return submissionRepository.save(submission);
    }
    public void deleteSubmission(Long id) {
        submissionRepository.deleteById(id);
    }
    public Submission updateSubmission(Long id, Submission updatedSubmission) {
```

```

Optional<Submission> optionalSubmission = submissionRepository.findById(id);
if (optionalSubmission.isPresent()) {
    Submission submission = optionalSubmission.get();
    submission.setTitle(updatedSubmission.getTitle());
    submission.setContent(updatedSubmission.getContent());
    submission.setUpdatedAt(LocalDateTime.now());
    return submissionRepository.save(submission);
}
throw new RuntimeException("Submission not found with ID: " + id);
}

public Page<Submission> getSubmissions(int page, int size, String status) {
    Pageable pageable = PageRequest.of(page, size);
    if (status != null && !status.isEmpty()) {
        return submissionRepository.findByStatus(status, pageable);
    }
    return submissionRepository.findAll(pageable);
}

public Submission approveSubmission(Long id) {
    Optional<Submission> optionalSubmission = submissionRepository.findById(id);
    if (optionalSubmission.isPresent()) {
        Submission submission = optionalSubmission.get();
        submission.setStatus("approved");
        submission.setApprovedAt(LocalDateTime.now());
        return submissionRepository.save(submission);
    }
    throw new RuntimeException("Submission not found with ID: " + id);
}

public Submission rejectSubmission(Long id) {
    Optional<Submission> optionalSubmission = submissionRepository.findById(id);
    if (optionalSubmission.isPresent()) {
        Submission submission = optionalSubmission.get();
        submission.setStatus("rejected");
        submission.setUpdatedAt(LocalDateTime.now());
        return submissionRepository.save(submission);
    }
    throw new RuntimeException("Submission not found with ID: " + id);
}
}

```

5.11 个人主页模块

5.11.1 个人主页模块界面设计

用户在二次元社区首页点击头像，可以进入个人主页，如图 5-23 所示。用户可以查看自己的个人信息，编辑个人资料。主页默认显示自己的投稿作品、收藏、历史记录。其次，用户可以设置自己的个人公告的展示，通过自主答题选择出自

己最喜欢的二次元角色之后，就会展示在个人中心的主页。同时用户可以选择进入投稿查看自己的全部作品，选择进入收藏页查看自己的收藏，选择进入历史记录页查看自己的历史记录。

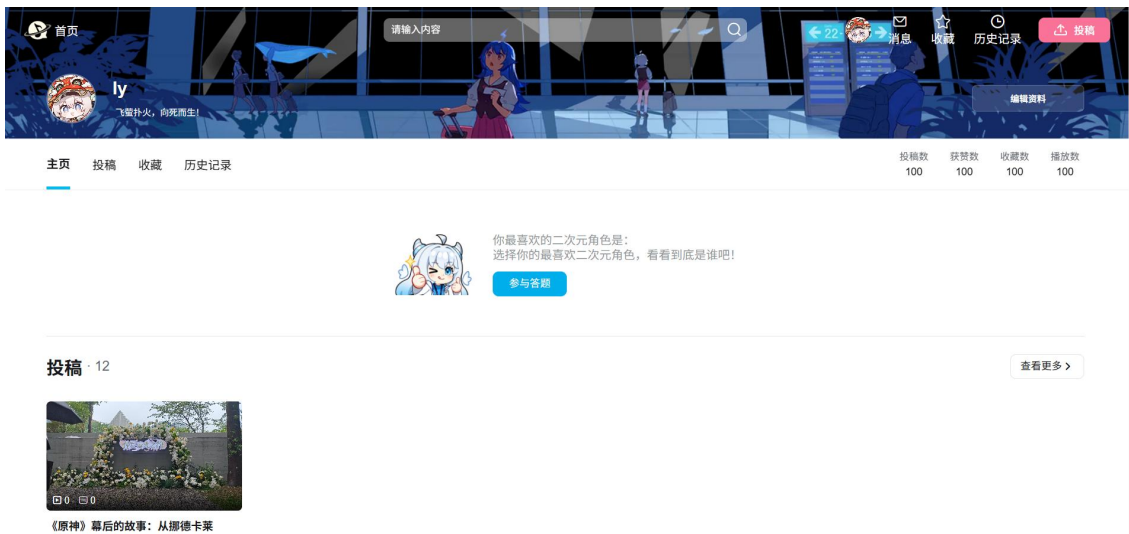


图 5-23 个人主页界面设计图

5. 11. 2 个人主页管理模块功能设计

用户点击编辑资料，弹出个人信息详情弹窗，用户编辑完个人资料后，点击确认即视为修改成功，流程图如图 5-24 所示。

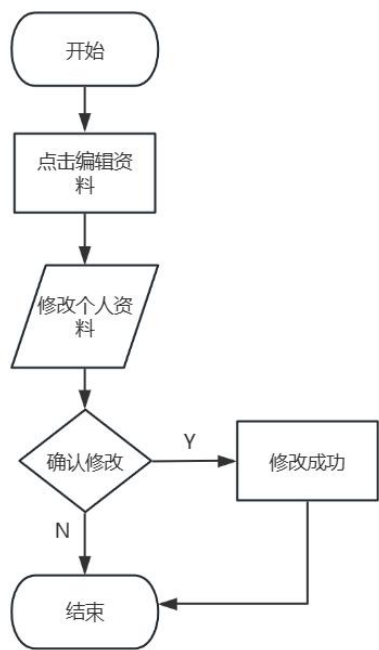


图 5-24 编辑个人资料流程图

5. 11. 3 个人主页模块核心代码

在个人主页模块，主要需要实现当用户修改个人信息的逻辑代码，以下是该功能的核心代码：

```

@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
    public User getUserProfile(Long userId) {
        Optional<User> optionalUser = userRepository.findById(userId);
        if (optionalUser.isPresent()) {
            return optionalUser.get();
        }
        throw new RuntimeException("User not found with ID: " + userId);
    }
    public User updateProfile(Long userId, User updatedUser) {
        Optional<User> optionalUser = userRepository.findById(userId);
        if (optionalUser.isPresent()) {
            User user = optionalUser.get();
            user.setEmail(updatedUser.getEmail());
            user.setAvatar(updatedUser.getAvatar());
            user.setBio(updatedUser.getBio());
            user.setUpdatedAt(LocalDateTime.now());
            return userRepository.save(user);
        }
        throw new RuntimeException("User not found with ID: " + userId);
    }
    public void deleteProfile(Long userId) {
        Optional<User> optionalUser = userRepository.findById(userId);
        if (optionalUser.isPresent()) {
            userRepository.deleteById(userId);
        } else {
            throw new RuntimeException("User not found with ID: " + userId);
        }
    }
}

```


6 系统功能测试

6.1 测试方法

本系统所有的功能测试都是采用黑盒测试方法，在测试时，不需要了解系统的内部代码结构，只需要检查程序是否能够按照预期实现效果。测试重点包括用户的注册登录、投稿作品、收藏管理、历史记录管理，以及管理员的用户管理、作品管理、评论管理、公告管理。测试过程将模拟典型用户场景，如浏览同人作品、个人创作分享、社区讨论参与等，同时验证系统的运行稳定性，确保系统功能符合需求规格，并具备良好的用户体验和二次元文化适配性。

6.2 测试环境

操作系统：Windows 11
JDK 版本：JDK 18
数据库：MySQL 5.5.22
Maven 版本：Maven 3.8.4
客户端：Google Chrome

6.3 功能测试

6.3.1 登录模块功能测试

在进行登录测试时，分为多种情况一一测试，确保系统功能能够正常实现。首先测试用户输入验证码的各种情况，测试用例及预期结果如表 6-1 所示。

表 6-1 验证码测试情况表

测试场景	测试用例	预期结果	测试结果
正确验证码	验证码=session 存储的值	通过验证	成功
错误验证码	验证码=" &&& "	提示验证码错误	成功
空验证码	验证码=" "	提示验证码不能为空	成功
验证码失效	过期验证码	提示验证码已过期	成功

用户在输入用户名和密码的各种情况如表 6-2 所示。

表 6-2 用户名密码测试情况表

测试场景	测试用例	预期结果	测试结果
正确用户名和密码	已注册的有效用户	通过验证	成功
不存在用户	未注册的用户名	提示用户不存在	成功
错误密码	正确的用户名+错误密码	提示密码错误	成功
空密码	密码为空	提示密码不能为空	成功

6.3.2 用户管理模块功能测试

管理员在进入用户管理模块后，默认显示用户表。测试管理员对用户表进行的各种操作，如表 6-3 所示。

表 6-3 用户管理模块测试情况表

测试场景	测试用例	预期结果	测试结果
基础模糊查询	昵称=" a"	通过验证	成功
删除一位用户	选择一位用户	提示验证码错误	成功
重置一位用户密码	选择一位用户重置密码	提示验证码不能为空	成功
批量删除用户	选择 10 位用户	返回成功，用户表减少 10 条	成功
批量重置用户密码	选择 20 位用户	返回成功，对应用户密码重置	成功

6.3.3 作品管理模块功能测试

管理员进入作品管理模块，默认显示已通过的作品，分别测试已通过和未通过的作品表。管理员可以对已通过的作品进行的各种操作，如表 6-4 所示。

表 6-4 已通过作品测试情况表

测试场景	测试用例	预期结果	测试结果
查看作品	选择一条作品	显示作品详情	成功
驳回一条作品	选择一条作品驳回	驳回成功，未通过增加一条作品	成功
批量驳回作品	选择 10 条作品	返回成功，未通过增加 10 条作品	成功
模糊查询作品	作品名=" aaa"	显示查询到的作品	成功

测试管理员可以对未通过的作品进行的各种操作，如表 6-5 所示。

表 6-5 未通过作品测试情况表

测试场景	测试用例	预期结果	测试结果
查看图文作品	选择一条图文作品	显示图文作品详情	成功
查看视频作品	选择一条视频作品查看	显示视频作品详情	成功
审核一条作品	选择一条作品审核	审核成功，已通过增加一条作品	成功
批量审核作品	选择 10 条作品审核	审核成功，已通过增加 10 条作品	成功
模糊查询作品	作品名=" bbb"	显示查询到的作品	成功

6.3.4 评论管理模块功能测试

管理员在评论管理模块可以进行的操作及其测试结果如表 6-6 所示。

表 6-6 评论管理模块测试情况表

测试场景	测试用例	预期结果	测试结果
基础模糊查询	评论内容=" qwq "	显示查询到的评论	成功
删除一条子评论	选择一条子级评论	提示删除成功，评论表减少一条	成功
删除一条父评论	选择一条父级评论	提示删除成功，评论表减少一条评论及其所有子评论	成功
批量删除评论（只包含子评论）	选择 20 条子评论	提示删除成功，评论表减少 20 条	成功
批量删除评论（包含父评论）	选择 10 条评论（包含 3 条父评论）	提示删除成功，评论表减少 7 条子评论和 3 条父评论及其所有子评论	成功
查看评论	查看一条评论	显示评论详情	成功

6.3.5 公告管理模块功能测试

管理员进入公告管理模块可以进行的操作及其测试结果如表 6-7 所示。

表 6-7 公告管理模块测试情况表

测试场景	测试用例	预期结果	测试结果
基础模糊查询	公告名称=" a "	显示查询到的公告	成功
删除一条公告	选择一条公告	返回成功，公告表减少一条	成功
添加一条公告	输入公告信息	返回成功，公告表增加一条	成功
修改一条公告	选择一条公告，修改信息	返回成功，公告信息改变为修改后的	成功
批量删除公告	选择 5 条公告	返回成功，公告表减少 5 条	成功

6.3.6 首页模块功能测试

用户在首页点击作品时，可以进行的操作及其测试结果如表 6-8 所示。

表 6-8 首页模块测试情况表

测试场景	测试用例	预期结果	测试结果
点击一个作品（用户未登录）	点击一个作品	弹出登录弹窗，提示用户登录	成功
点击一个作品（用户已登录）	点击一个作品	进入作品详情页	成功

6.3.7 消息模块功能测试

用户点击消息时，需要分别对四种情况进行测试，可以进行的操作及其测试结果如表 6-9 所示。

表 6-9 消息模块测试情况表

测试场景	测试用例	预期结果	测试结果
点击消息（用户未登录）	点击消息	弹出登录弹窗，提示用户登录	成功
点击消息（用户已登录）	点击消息	进入收藏页	成功
点击评论我的	点击评论我的	显示评论我的消息	成功
点击回复我的	点击回复我的	显示回复我的消息	成功
点击收到的赞	点击收到的赞	显示所有作品收到的赞	成功
点击系统消息	点击系统消息	显示系统消息	成功

6.3.8 收藏模块功能测试

当用户浏览一条作品的作品详情时，可以点击收藏，当用户点击进入收藏页后可以进行的操作及其测试结果如表 6-10 所示。

表 6-10 收藏模块测试情况表

测试场景	测试用例	预期结果	测试结果
点击收藏（用户未登录）	点击收藏	弹出登录弹窗，提示用户登录	成功
点击收藏（用户已登录）	点击收藏	进入收藏页	成功
添加收藏夹	输入收藏夹信息	收藏夹增加一条	成功
取消一条收藏	选择一条收藏取消	收藏表减少一条	成功
移动一条收藏	选择一条收藏移动至 xx 收藏夹	xx 收藏夹中增加一条，原收藏夹减少一条	成功

6.3.9 历史记录模块功能测试

当用户浏览一个作品就会产生一条历史记录，当用户点击历史记录时，进入到历史记录页可以进行的操作及其测试结果如表 6-11 所示。

表 6-11 历史记录模块测试情况表

测试场景	测试用例	预期结果	测试结果
点击历史记录（用户未登录）	点击历史记录	弹出登录弹窗，提示用户登录	成功
点击历史记录（用户已登录）	点击历史记录	进入历史记录页	成功
删除一条历史记录	选择一条历史记录删除	历史记录表减少一条	成功
全部清空历史记录	点击全部清空按钮	历史记录表减少该用户全部历史记录	成功

6.3.10 投稿模块功能测试

用户点击投稿时，进入投稿详情页可以进行的操作及其测试结果如表 6-12 所示。

表 6-12 投稿模块测试情况表

测试场景	测试用例	预期结果	测试结果
点击投稿（用户未登录）	点击投稿	弹出登录弹窗，提示用户登录	成功
点击投稿（用户已登录）	点击投稿	进入投稿详情页	成功
投稿一条图文作品	输入图文作品信息	作品表增加一条	成功
投稿一条视频作品	输入视频作品信息	作品表增加一条	成功

6.3.11 个人主页模块功能测试

用户点击头像，进入个人主页时，可以进行的操作及其测试结果如表 6-13 所示。

表 6-13 个人主页模块测试情况表

测试场景	测试用例	预期结果	测试结果
点击头像（用户未登录）	点击投稿	弹出登录弹窗，提示用户登录	成功
点击头像（用户已登录）	点击投稿	进入个人主页	成功
编辑个人资料	修改个人资料	用户表该用户信息改变为修改后的	成功
查看投稿	点击投稿	显示用户自己的投稿作品	成功
查看收藏	点击收藏	显示用户自己的收藏	成功
查看历史记录	点击历史记录	显示用户自己的历史记录	成功

7 总结

本项目设计初衷旨在为二次元爱好者提供一个便捷、高效且充满活力的社区空间。项目初期通过深入的需求分析，明确了用户的核心需求，包括但不限于用户注册登录、投稿作品、评论点赞等功能，并确定了系统的一些针对于二次元用户的功能需求，如自由选择自己最喜欢的二次元 IP、查看自己今日观看的二次元 IP 喜爱排行榜等等。在系统设计阶段，选择了微服务架构，用来增强系统的可扩展性和维护性，并由此选择 Spring Boot 作为核心框架，得益于其简化配置、提高开发效率的优势。数据库设计方面，精心构建了多个实体模型来支持上述功能需求，确保数据的有效组织与存储。技术选型上，前端直接运用了 Vue.js 框架，因其易于上手和组件化的特性非常适合本项目的 UI 需求；而后端除了 Spring Boot 外，还引入了 MyBatis 用于简化数据库操作，以及 Redis 来提升系统性能。测试环节，考虑几乎所有可能的情况去进行代码测试，确保了代码的质量和系统的稳定性。最终，本项目成功完成，实现了预期效果，为二次元爱好者搭建了一个理想的社交平台。

个人而言，通过此次项目，我深刻体会到了从需求分析到系统测试的全过程，学习到了许多前沿的技术知识，如微服务架构的设计原则、Spring Boot 和 Vue 结合开发等。更重要的是，我学会了如何解决实际开发中遇到的各种问题，增强了解决问题能力和项目管理能力。在开发设计的过程中，我明白了一句话，即“知之者不如好之者，好之者不如乐之者”，说的是热爱可以大于一切，这是我自己能坚持下来的动力来源。未来，计划进一步优化系统性能，增加更多个性化推荐算法，进一步提高二次元用户的理想化体验。

参考文献

- [1]孔祥盛,马玉磊.Java Web 基础与实例教程[M].北京:人民邮电出版社,2023.
- [2]徐家喜,王小正,朱杰.Java EE 框架技术与案例教程[M].南京:南京大学出版社,2023.
- [3]李兴华,马云涛.Spring 开发实战[M].北京:人民邮电出版社,2023.
- [4]陈洋.基于 B/S 架构的智慧化工园区管理系统设计与实现[D].南京:南京信息工程大学,2024.
- [5]柳伟卫.Vue.js+Spring Boot 全栈开发实战[M].北京:人民邮电出版社,2023.
- [6]陈承欢.Vue.js 基础与应用开发实战[M].北京:人民邮电出版社,2022.
- [7]温谦.JavaScript+Vue.js Web 开发案例教程[M].北京:人民邮电出版社,2022.
- [8]卢长鹏,张业男,周敏.Java 程序设计项目化教程[M].北京:化学工业出版社,2024.
- [9]张桓,徐丽.Java 语言程序设计与实现[M].北京:人民邮电出版社,2023.
- [10]舒蕾,刘均.MySQL 数据库应用与维护项目式教程[M].北京:人民邮电出版社,2023.
- [11]刘守业.基于深度学习的就业推荐平台研究与应用[D].太原:太原师范学院,2024.
- [12]高毓聪,冯宁,邱斐.低代码平台在工程管理中的应用与实践[J].中国建设信息化,2025,(04):64-68.
- [13]阳皓筠.基于 JavaWeb 的视频采集与传输系统设计[J].现代信息科技,2025,9(04):107-111.
- [14]韦珍娜,陈宇佳.基于 Springboot 的服装租赁系统设计[J].电脑编程技巧与维护,2025,(01):35-38.
- [15]余波.基于 SpringBoot 的高职院校竞赛管理系统研究[J].九江学院学报(自然科学版),2024,39(04):70-74+113.
- [16]赵叶平,陈圣,马军,等.基于 Spring Boot+VUE+Uni-app 框架的文明城市智慧管理系统设计与实现[J].工业控制计算机,2025,38(04):139-140+153.
- [17]高键,王海淼.基于 SpringBoot 的结构化面试一体化系统的设计与实现[J].电脑知识与技术,2024,20(33):47-50.

- [18]殷志强, 钟卫东. 基于 SpringBoot 应用中的 XSS 攻击防御技术[J]. 工业控制计算机, 2024, 37(11):62-64.
- [19]安宇鑫. 基于 SpringBoot+Vue 的教学评价专家抽课系统的设计与实现[J]. 电脑知识与技术, 2024, 20(32):29-34.
- [20]朱静. 基于 SpringBoot 和 Vue 的全能型供电所管理系统设计与实现[D]. 上海: 华东师范大学, 2024.
- [21]Davi V . Java Real World Projects:A pragmatic guide for building modern Java applications (English Edition) [M]. sIndia:BPB Publishers, 2024.

致谢

在我开发设计本系统的这段时间，离不开我的同学朋友帮我思考项目需求，离不开指导老师的悉心教导，离不开家人们的贴心照顾，让我能够顺利完成项目。其中，我要感谢为我提供巨大帮助的李娟老师，是她多次辅导我论文该怎么写，在论文写作上给予了我非常细致的指导，同时在为我解决各种问题时也非常耐心。然后，我也要感谢朱安老师对我的系统设计提供的技术支持，在我不知道的专业问题中提供了帮助，尤其是在数据库设计与优化方面，解答了我很多疑问，帮助了我攻克许多代码难题，让我在开发项目的过程中能够顺利进行下去。同时，我也要感谢我的家人，是他们在背后的默默支持和贴心照顾，让我能够全身心地投入到项目中。每当我调试代码或修改论文忘记吃饭时，你们的关心和照顾是我坚持下去的动力。最后，我要感谢所有计算机科学与技术专业的老师们，你们的多年教导和认真负责的教学精神，使我真的学到了这些知识并运用它们到实际工作中，无论是课堂上的理论知识，还是实践中的技术应用，你们的教诲都让我学到了很多。最重要的是让我从一名对编程充满好奇的学生成长为能够独立完成项目的开发者。我衷心祝愿你们未来工作顺顺利利，身体健健康康，每天开开心心！